



Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y

Eléctrica

Materia: Percepción

Docente: Susana Viridiana

Gutiérrez Martínez

**PROYECTO INTEGRADOR DE
APRENDIZAJE (PIA)**

**REPORTE TÉCNICO;
ROBOT DE FÚTBOL
TELEOPERADOR POR
PERCEPCIÓN
MIOELÉCTRICA (EMG)**

Grupo: 002 **Hora:** M4-M6

Día: martes



Integrantes del equipo:

Nombre completo	Matrícula	Carrera
Valeria Abigail González Sada	1963075	IMTC

Semestre: Agosto - diciembre 2025

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 05 diciembre 2025

CONTENIDO

RESUMEN.....	4
INTRODUCCIÓN.....	5
OBJETIVOS DEL PROYECTO.....	5
2.1. Objetivo General.....	5
2.2.2. Objetivos Específicos.....	5
MARCO TEÓRICO.....	6
3.1. Sensores y Actuadores Utilizados.....	6
3.1.1. Sensor de Percepción: AD8232 (EMG).....	6
3.1.2. Actuadores de Movimiento: Motores DC TT.....	6
3.1.3. Actuadores de Pateo.....	7
3.1.4. Componentes de Procesamiento y Potencia.....	7
3.1.4. Algoritmos de Control Implementados.....	9
Sistemas de Lógica Difusa.....	9
3.1.5. Diagrama del sistema de percepción.....	11
3.8. Tecnologías Relacionadas.....	11
3.8.1. Comunicación Inalámbrica: vía Wi-Fi (AP/STA con UDP),.....	11
3.8.2. Lógica difusa.....	11
METODOLOGÍA.....	12
4.1. Diseño Mecánico (CAD).....	12
4.1.2. Ensamblaje Físico y Modificaciones.....	13
4.2. Diseño Electrónico (Diagramas).....	14
4.2.1. Sistema Emisor (Percepción).....	16
4.2.2. Sistema Receptor (Robot).....	17
4.3. Diseño de Control y Percepción (Lógica Difusa).....	18
7.1.1. Metodología de Ajuste.....	41
7.1.2. Mapeo de Señal de Control.....	41
5.RESULTADOS.....	42
5.1. PARTE MECÁNICA.....	42
A continuación, se muestran evidencias de esta parte de mecánica:.....	43
5.2. ETAPA ELECTRÓNICA Y CONTROL.....	44
5.4. ANÁLISIS DE LAS PRUEBAS FINALES.....	59

5.5. EVIDENCIAS GRÁFICAS DEL PROTOTIPO	66
.....	66
5.5. Análisis del Sistema Difuso (casos) (5)	68
5.6. PRUEBAS DE FUNCIONAMIENTO (VIDEO)	71
5.8. Código Fuente	71
Implementación de Firmware (Códigos Fuente)	71
7.3.1. Firmware del Emisor (Procesamiento Difuso).....	73
6.CONCLUSIONES.....	74
6.1. Valeria Abigail González Sada 1963075	74
SUGERENCIAS FUTURAS.....	74
COMENTARIOS PERSONALES.....	75
8.1. Valeria Abigail González Sada 1963075	75
ANEXOS	76
9.1. (A). Repartición de Tareas	76
1. Valeria Abigail González Sada 1963075	76
9.2. (A) Pseudocódigo 1 – Ploteo de motores (Python)	76
PSEUCÓDIGO 2 – graficos 2D EMG A PWM.....	80
9.2 ©. CÓDIGO FINAL DEL EMISOR.....	83
9.2 (D) – CÓDIGO FINAL DEL RECEPTOR.....	90
1	99
REFERENCIAS DE APOYO	104

1. RESUMEN

El proyecto se centró en el desarrollo de un sistema de teleoperación gestual basado en señales electromiográficas (EMG) y lógica difusa, con el objetivo general de traducir la actividad muscular del usuario en comandos direccionales para un robot móvil. La propuesta buscó demostrar la viabilidad de integrar percepción biológica, procesamiento inteligente y ejecución robótica en un ciclo completo de interacción humano-máquina, con aplicaciones potenciales en rehabilitación, asistencia remota y control inclusivo de dispositivos.

La metodología utilizada combinó varias etapas: primero, la adquisición de señales EMG mediante electrodos conectados a sensores EMG8232, lo que permitió capturar la actividad muscular real del usuario; posteriormente, el diseño de funciones de pertenencia utilizando la librería *scikit-fuzzy* (*skfuzzy*), donde se definieron funciones trapezoidales en las esquinas y triangulares en la región central, permitiendo asignar grados de pertenencia a diferentes gestos musculares; y finalmente, la implementación de un sistema de comunicación inalámbrica WiFi WLAN entre dos módulos ESP32, uno actuando como emisor y otro como receptor. Este enfoque modular permitió probar cada componente de manera independiente antes de integrarlos en el prototipo final.

El emisor se encargó de la percepción y procesamiento de las señales provenientes de dos sensores EMG8232, uno por cada brazo del usuario, de modo que cada sensor controlaba de manera independiente un motor del carrito. La lógica difusa, implementada con reglas definidas en *skfuzzy*, determinó la acción dominante entre los posibles estados: adelante, atrás y paro. Solo cuando la decisión cambiaba respecto al estado anterior se enviaba el comando al receptor, optimizando la transmisión y evitando saturación de datos. El receptor, por su parte, interpretó los comandos recibidos y los tradujo en movimientos físicos mediante el controlador de motores DRV8833, ajustando la dirección y velocidad de los motores según la instrucción.

Los resultados principales demostraron que el sistema es capaz de interpretar gestos musculares capturados con electrodos de manera precisa y traducirlos en movimientos robóticos coherentes y estables. Las pruebas confirmaron la robustez de la lógica difusa para manejar variaciones suaves en la señal, la eficiencia del protocolo WiFi WLAN para comunicación directa y la capacidad del robot de ejecutar acciones en tiempo real. En conjunto, el proyecto validó un prototipo funcional, adaptable y con potencial de

aplicación en entornos biomédicos y tecnológicos, cumpliendo los objetivos planteados.

2. INTRODUCCIÓN

En el campo de la robótica moderna, el desarrollo de interfaces humano-máquina (HMI) intuitivas representa uno de los mayores desafíos. Tradicionalmente, la teleoperación de robots se ha basado en controles manuales como joysticks o teclados. Sin embargo, estos métodos carecen de una conexión natural e inmersiva entre el operador y la máquina.

Este proyecto aborda dicho desafío explorando un mecanismo de percepción avanzado: la electromiografía (EMG). A diferencia de los sensores inerciales que miden el movimiento físico, los sensores EMG detectan la actividad eléctrica generada por los músculos, permitiendo interpretar la intención de movimiento del operador antes de que este se ejecute por completo.

Aplicar esta tecnología en un entorno competitivo, como un partido de fútbol robótico, nos permite validar la viabilidad y robustez de un sistema de percepción mioeléctrico para el control en tiempo real, abriendo puertas a aplicaciones más complejas en prótesis biónicas, exoesqueletos de asistencia y control robótico de alta precisión.

2.1. OBJETIVOS DEL PROYECTO

2.1. Objetivo General

Documentar, analizar y presentar un robot de fútbol teleoperado, controlado por un sistema de percepción mioeléctrico basado en Lógica Difusa, integrando los conocimientos del curso para establecer una interfaz humano-máquina funcional y de baja latencia.

2.2.2. Objetivos Específicos

- Diseñar y construir un chasis robótico funcional que cumpla con las especificaciones de competencia (dimensiones 10x10x10 cm, peso < 750g).
- Implementar un sistema de percepción capaz de interpretar la intención de movimiento del operador, utilizando los sensores EMG AD8232 posicionados en las extremidades superiores.
- Desarrollar un algoritmo de Lógica Difusa que traduzca las señales mioeléctricas procesadas en comandos de actuación proporcionales para los motores del robot.

- Establecer un canal de comunicación inalámbrico robusto y de baja latencia entre el sistema de percepción (emisor) que es el exoesqueleto y el robot (receptor) utilizando el protocolo ESP-NOW- usando el microcontroladores ESP32.
- Validar la funcionalidad, precisión e intuición del prototipo en un escenario de simulación de partido de fútbol.

3. MARCO TEÓRICO

3.1. Sensores y Actuadores Utilizados

3.1.1. Sensor de Percepción: AD8232 (EMG)

El mecanismo de percepción seleccionado es el sensor de electromiografía (EMG) basado en el chip AD8232. Este sensor es un front-end analógico diseñado para medir la actividad eléctrica generada por el tejido muscular (potenciales de acción). A diferencia de los sensores inerciales que miden el movimiento físico, el AD8232 permite capturar la intención de movimiento del operador. En este proyecto, se utilizan dos de estos sensores para monitorear de forma independiente la activación muscular de ambos brazos, sirviendo como las entradas principales de nuestro sistema de control por percepción.



Ilustración 1. SENSOR EMG 8832

3.1.2. Actuadores de Movimiento: Motores DC TT

Para la tracción del robot se emplean dos motores DC con caja reductora de plástico, comúnmente conocidos como "motores TT". Estos motores son una solución estándar en robótica móvil de pequeña escala debido a su balance entre costo, torque y velocidad. Su alta tasa de reducción les permite mover el chasis con agilidad, y su control se realiza mediante modulación por ancho de pulso (PWM) a través de un driver de etapa de potencia.



Ilustración 2. MOTORES TT (RECEPTOR)

3.1.3. Actuadores de Pateo

Por decisión de diseño y para cumplir con las restricciones de peso y espacio, el prototipo no incluye un mecanismo de pateo activo (como un solenoide o servomotor). El movimiento del balón se logra por contacto directo con el chasis del robot.

3.1.4. Componentes de Procesamiento y Potencia

3.1.4.1. Tarjeta de Procesamiento: ESP32

El cerebro del sistema está compuesto por dos microcontroladores ESP32. Se eligió esta plataforma por dos razones principales: su potente procesador de doble núcleo, capaz de ejecutar la lógica de control y procesar las señales de los sensores simultáneamente; y su conectividad Wi-Fi y Bluetooth integrada. Se utiliza un ESP32 como "emisor" (conectado a los sensores EMG) y un segundo ESP32 como "receptor" (montado en el robot), permitiendo una arquitectura de sistema distribuida.



Ilustración 3. ESP32 (EMISOR Y RECEPTOR)

3.1.4.2. Etapa de Potencia: Driver DRV8833

Dado que los pines de un microcontrolador no pueden suministrar la corriente necesaria para los motores, se utiliza un driver de etapa de potencia. Se seleccionó el driver DRV8833, un puente-H dual de alta eficiencia. La elección de este componente fue crítica: su amplio rango de voltaje de operación (2.7V a 10.8V) lo hace compatible tanto con la fuente de alimentación de 9V como con los motores TT (que operan óptimamente entre 3-9V). Además, su alta eficiencia y encapsulado compacto son superiores al

L298N, siendo una elección deliberada para un sistema optimizado en energía y espacio.



Ilustración 4. DRIVER 8833

3.1.3.4. Fuente de Alimentación: Batería 9V

La restricción de diseño más significativa fue el volumen máximo de 10x10x10 cm. Esta limitación hizo inviable el uso de fuentes de poder voluminosas como packs de baterías AA o LiPo de gran capacidad. La solución fue utilizar una batería alcalina estándar de 9V (tipo 6LR61) que en este caso fueron 4 baterías de este tipo que ofrece una excelente densidad de energía para su reducido tamaño. Aunque las baterías de 9V son conocidas por su baja capacidad de entrega de corriente, la elección del driver DRV8833 de alta eficiencia fue clave para hacer viable esta solución energética, priorizando la compactación del diseño. Además, para la alimentación del esp del receptor (carrito) se utilizaron 3 baterías de 1.5 V donde tres baterías de 1.5 V en serie entregan 4.5 V, justo dentro del rango ideal de entrada del ESP32. Ese voltaje permite al regulador interno generar 3.3 V estables sin riesgo de sobrecarga ni fallos.



Ilustración 5. PILAS DE ALIMENTACIÓN (RECEPTOR)

3.1.3.4.1. Módulo de alimentación para protoboard

El módulo de alimentación para protoboard es un dispositivo diseñado para facilitar la conexión y regulación de energía en proyectos electrónicos. Se coloca directamente sobre la protoboard y permite suministrar voltajes estables, comúnmente de 3.3V y 5V, a los

diferentes componentes conectados. Su función principal es convertir la energía proveniente de una fuente externa —como una pila de 9V o un adaptador— en niveles seguros y adecuados para circuitos integrados, sensores y microcontroladores, evitando daños por sobrevoltaje y simplificando el cableado.

En el caso del carrito controlado con ESP32, este módulo cumple un papel esencial al recibir la energía de la pila de 9V y regularla para alimentar al microcontrolador, que actúa como receptor de las señales de control. Gracias a esta regulación, el ESP32 puede operar de manera confiable sin riesgos de fluctuaciones de voltaje, asegurando que el sistema de comunicación y control del carrito funcione correctamente. Además, el módulo facilita la distribución de energía hacia otros componentes del prototipo, como motores o sensores, manteniendo la organización y estabilidad del circuito.



Ilustración 6. Módulo de alimentación para proto (receptor-esp32)

3.1.4. Algoritmos de Control Implementados

3.1.4.1. Sistemas de Lógica Difusa

La lógica difusa (Fuzzy Logic) es una metodología de control que permite manejar información imprecisa o con ruido, imitando la forma en que razona el cerebro humano. A diferencia de la lógica booleana tradicional, que solo admite valores binarios (verdadero = 1 o falso = 0), la lógica difusa introduce el concepto de grados de verdad, lo que significa que una variable puede tomar valores intermedios entre 0 y 1. Esto ofrece un marco más flexible para representar fenómenos reales que no pueden describirse únicamente con estados absolutos.

En el contexto de este proyecto, el uso de un controlador difuso resulta esencial debido a la naturaleza de las señales electromiográficas (EMG). Estas señales biológicas presentan variaciones no lineales, ruido y efectos de fatiga muscular que harían inestable un control tradicional tipo “On/Off”. El controlador difuso permite suavizar estas entradas, transformando la intensidad de la contracción muscular en una señal PWM

(Pulse Width Modulation) continua y fluida. De esta manera, los actuadores del vehículo responden de forma más estable y precisa a las órdenes musculares del usuario.

El sistema de control propuesto se diseñó bajo un esquema MISO (Multiple Input, Multiple Output) desacoplado, pensado para un vehículo de tracción diferencial. Las entradas corresponden a dos sensores de mioelectrografía, uno colocado en el lado izquierdo y otro en el derecho, con un rango de lectura de 0 a 4095 gracias a la resolución de 12 bits del ADC. La variable lingüística asociada a estas entradas es la intensidad de contracción. Por otro lado, las salidas corresponden a dos motores de corriente continua, cuya velocidad de rotación se controla mediante un ciclo de trabajo PWM que varía entre 0% y 100%.

Para granularizar el control y obtener una respuesta más precisa, se definieron cinco conjuntos difusos tanto para las entradas como para las salidas. Se emplearon funciones de membresía de tipo triangular y trapezoidal que cubren todo el rango operativo. Las etiquetas lingüísticas establecidas son: *Reposo* (ausencia de señal o ruido base), *Muy Baja* (contracción leve, inicio de movimiento), *Media* (contracción estándar de operación), *Alta* (contracción fuerte para velocidad rápida) y *Hulk* (saturación del sensor o fuerza máxima, modo turbo). Estas categorías permiten que el sistema traduzca las señales musculares en acciones proporcionales y adaptativas sobre los motores del vehículo.



ideal para controlar dispositivos que requieren precisión y flexibilidad.

En el caso del control de un carrito robótico mediante ambos brazos, la lógica difusa permite interpretar las señales de los sensores EMG8832, que captan la actividad muscular, y convertirlas en comandos proporcionales para el movimiento del robot. El ESP32 actúa como unidad de procesamiento, aplicando reglas difusas para determinar la dirección y velocidad del carrito según la intensidad y combinación de movimientos detectados en cada brazo. Esto ofrece un control más natural y adaptable, reduciendo errores y mejorando la experiencia del usuario en aplicaciones de robótica asistida o rehabilitación.

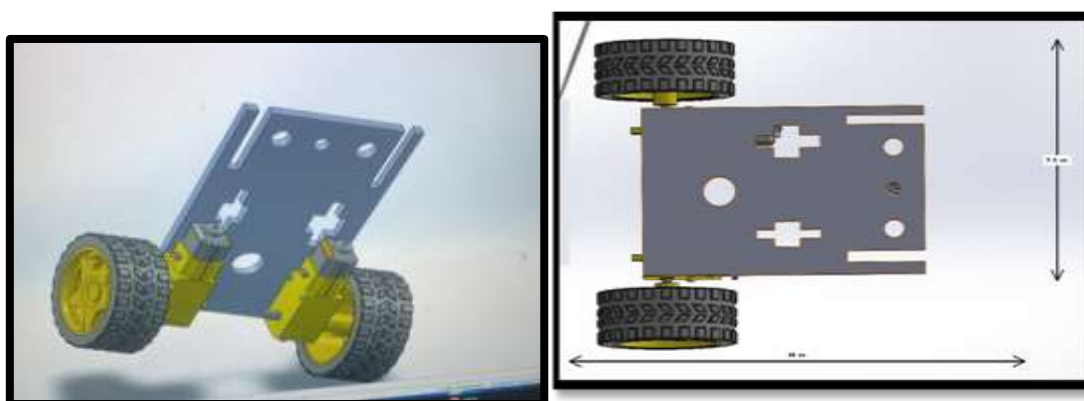
3. METODOLOGÍA

El desarrollo del proyecto se dividió en tres áreas de trabajo principales que corrieron en paralelo: el diseño mecánico y estructural del chasis, el diseño electrónico para la interconexión de componentes, y el diseño del sistema de control y percepción.

4.1. Diseño Mecánico (CAD)

El diseño mecánico, partió de un chasis 2WD de acrílico estándar, que tuvo que ser modificado para cumplir con las estrictas especificaciones de 10x10x10 cm. El material principal es el acrílico, seleccionado por su ligereza y facilidad de modificación. El cual se realizó en el software de Solidworks como se muestra a continuación.

En la Figura 1 se presentan las capturas del diseño CAD, donde se validaron las dimensiones finales.



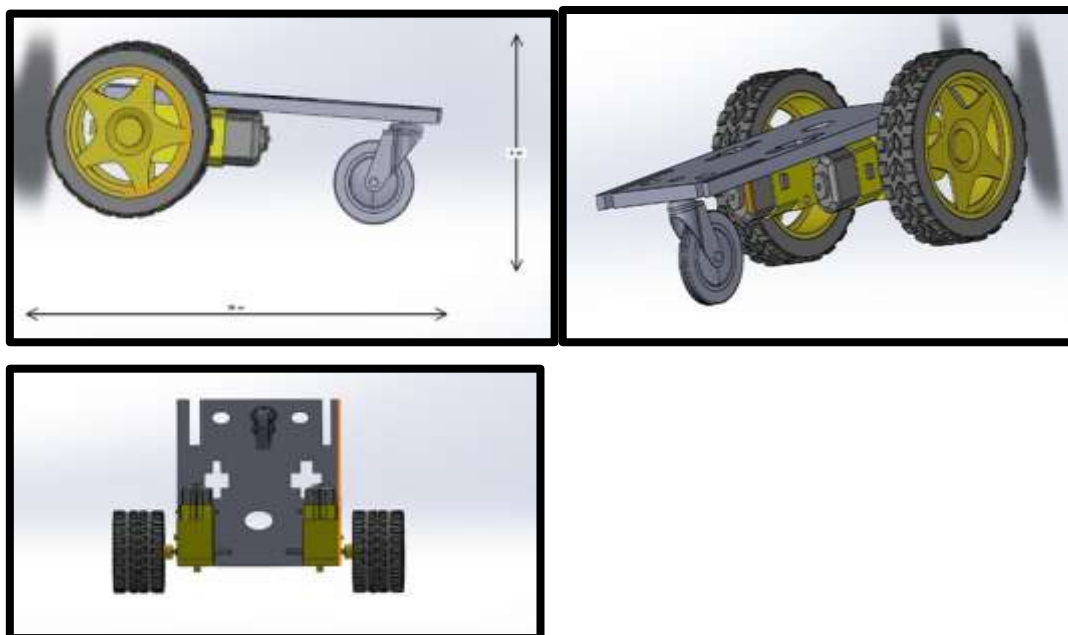


Ilustración 8. DISEÑO CAD RECEPTOR (CARRITO). VISTAS SUPERIOR, INFERIOR, ISOMETRICA Y LATERAL

El diseño CAD del carrito robótico muestra una estructura compacta y funcional, pensada para facilitar la integración de sensores, electrónica y motores en un chasis estable. En las vistas superior y lateral se validan las dimensiones finales de la base, asegurando espacio suficiente para los módulos ESP32, el controlador DRV8833 y las conexiones de los sensores EMG. La distribución simétrica de las ruedas principales y la rueda loca trasera garantiza estabilidad en desplazamientos direccionales, mientras que la placa metálica superior permite fijar componentes sin interferir con el movimiento.

Las capturas también evidencian una planificación mecánica orientada a la modularidad: el diseño contempla accesos laterales para cableado y ventilación, así como una altura adecuada para evitar interferencias con el suelo. La elección de materiales y proporciones sugiere un equilibrio entre ligereza y rigidez estructural, ideal para pruebas de teleoperación gestual. Este modelo CAD valida no solo la geometría del prototipo, sino también su viabilidad para integrar control difuso, comunicación inalámbrica y respuesta motriz en tiempo real.

4.1.2. Ensamblaje Físico y Modificaciones

El proceso de ensamblaje físico, se describe a continuación:

1. Primero, se cortó el chasis original de acrílico con un cortador de acrílico para ajustarlo a las medidas exactas del proyecto.
2. Después, con tornillos de 3 mm, se colocaron los dos motores TT en la parte

inferior del chasis.

3. Se realizó una pequeña muesca en la base de la rueda loca (caster) para que no interviniera con la carcasa de los motores.
4. Se atornilló la rueda loca en la parte frontal inferior, opuesta a los motores.
5. Finalmente, se atornillaron las llantas de 66x27mm a los ejes de los motores de cada lado.

La Figura 2 muestra una evidencia fotográfica de este proceso de ensamblaje manual



Ilustración 9. ENSAMBLAJE FÍSICO (RECEPTOR)-CARRITO

4.2. Diseño Electrónico (Diagramas)

El diseño electrónico del proyecto, se basa en una arquitectura de dos sistemas separados: un "Sistema Emisor" (el exoesqueleto de percepción) y un "Sistema Receptor" (el robot). La comunicación entre ambos es inalámbrica.

El diagrama electrónico completo del proyecto, realizado en KiCad, se presenta en la Figura 3. Este diagrama cumple simultáneamente con los requisitos de ser un bosquejo del proyecto (mostrando los componentes clave como sensores, procesadores, etapa de potencia, actuadores y batería) y un diagrama detallado que especifica las conexiones a pines.

El sistema se compone de un ESP32 emisor conectado a dos módulos EMG: en el EMG derecho se enlaza 3V del ESP32 a 3.3V del EMG, GND a GND, D16 a LO+, D17 a LO- y VN del ESP32 al OUTPUT del EMG; en el EMG izquierdo se conecta igualmente 3V a 3.3V y GND a GND, mientras que LO+ del EMG va a D14 y LO- a D4 del ESP32, con el OUTPUT del EMG dirigido a VP del ESP32 y las señales hacia la protoboard. En el carrito receptor, la etapa de potencia recibe 5V hacia Vin del ESP32 y GND común con todo el sistema, mientras que las salidas PWM del ESP32 (3.3V) se conectan al driver 8833 en sus secciones A y B, además de alimentar la entrada STDBY del mismo; la batería de 9V de motores se conecta a Vin del driver y su GND se une al GND del sistema.

De esta forma, los motores se alimentan con la batería de 9V, mientras que el ESP32 receptor obtiene energía estable mediante un módulo de alimentación para protoboard conectado a otra pila de 9V, integrando así sensores EMG, microcontroladores y etapa de potencia en un esquema electrónico funcional.

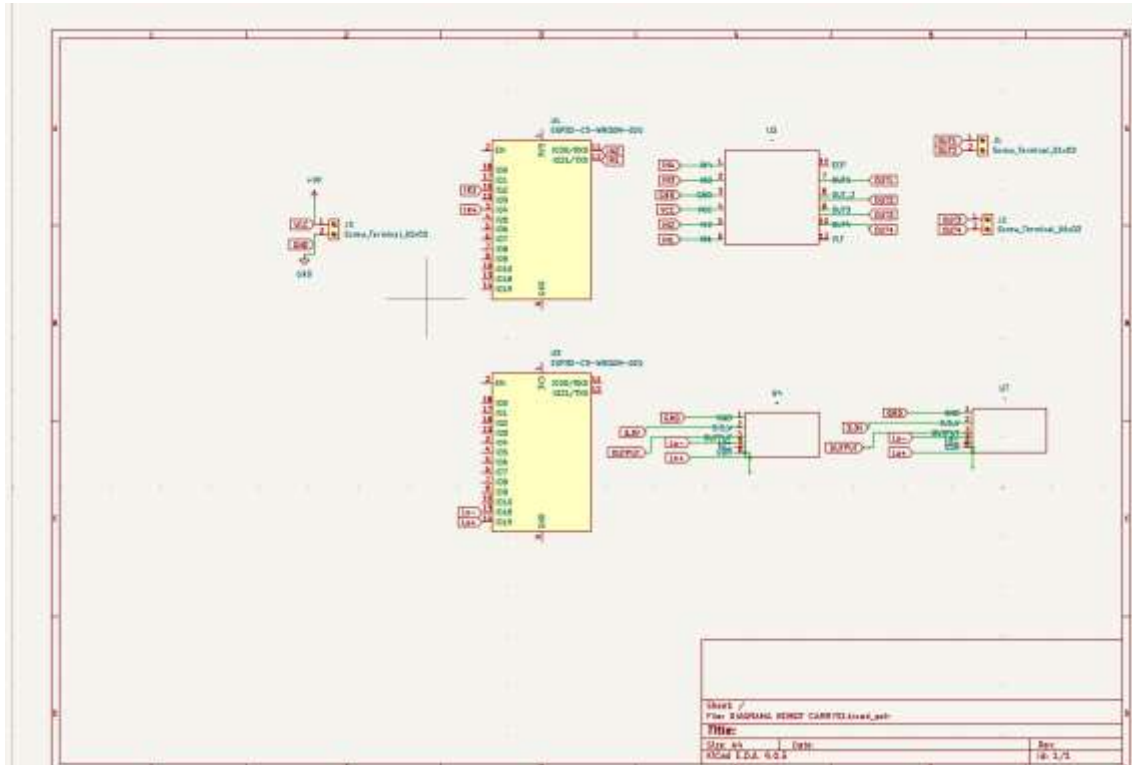


Ilustración 10. ESQUEMÁTICO ELECTRÓNICO

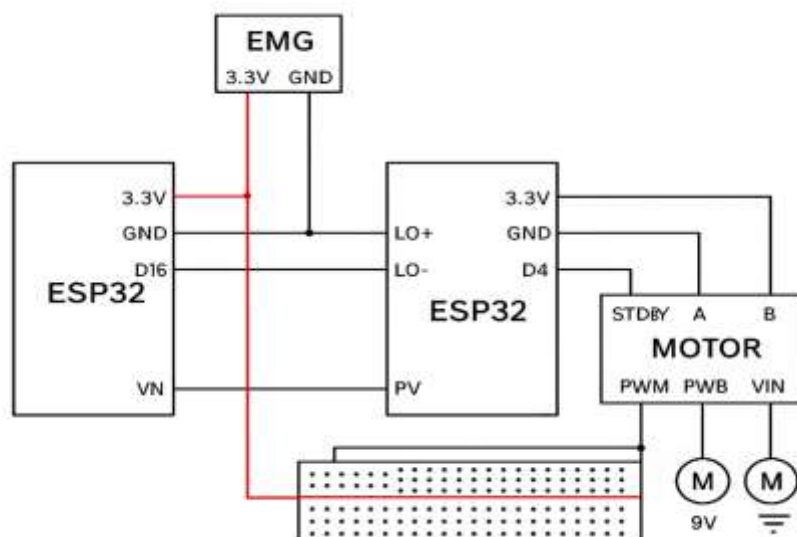


Ilustración 11. ESQUEMÁTICO

4.2.1. Sistema Emisor (Percepción)

El sistema emisor está conformado por un ESP32 configurado como servidor de datos, encargado de recibir y procesar la actividad muscular proveniente de dos sensores EMG. El primero, un EMGAD8232, se conecta al pin digital D2 para la señal baja (Lo-) y a Lo+ para la señal alta, mientras que su salida principal se dirige al pin D36. Este módulo permite capturar la actividad muscular de un brazo y transformarla en información que será interpretada mediante un algoritmo de control difuso, suavizando las variaciones de la señal antes de enviarlas al receptor.

El segundo sensor, EKG2, se integra al ESP32 con una conexión específica: 3.3 V al pin 3V3, GND a tierra, OUTPUT al pin VN (ADC1_CH3), LO+ al pin D16 y LO- al pin D17. Este segundo canal permite capturar la actividad muscular del otro brazo, logrando que cada sensor controle de manera independiente un motor del carrito. De esta forma, el sistema obtiene dos entradas biomédicas diferenciadas que enriquecen la interpretación de los gestos y mejoran la precisión del control.

Ambos sensores trabajan en conjunto con el algoritmo de lógica difusa implementado en la librería scikit-fuzzy, que asigna funciones de pertenencia trapezoidales en las esquinas y triangulares en el centro para definir los estados de movimiento: adelante, atrás y paro. El ESP32 transmite únicamente cuando se detecta un cambio significativo en el gesto, optimizando la comunicación inalámbrica vía WiFi WLAN y evitando saturación de datos. Así, el sistema logra transformar señales musculares en comandos suaves y estables que permiten al carrito ejecutar movimientos en tiempo real.

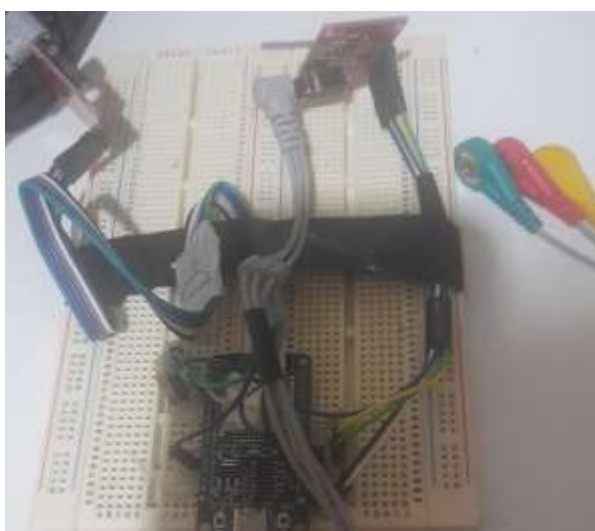


Ilustración 12. EMISOR (CONTROL) CON SENSORES EMG

4.2.2. Sistema Receptor (Robot)

El sistema receptor consiste en el robot del carrito que recibe los comandos enviados por el emisor y los procesa mediante un algoritmo de control difuso. La alimentación se organiza en dos etapas: una batería de 9 V dedicada a los motores, conectada al driver DRV8833, y un módulo de alimentación para protoboard que también se alimenta de una pila de 9 V y regula la energía para el microcontrolador ESP32 del carrito. Esta separación asegura que los actuadores reciban la potencia necesaria mientras la lógica de control del ESP32 se mantiene estable y protegida contra interferencias.

El driver DRV8833 funciona como puente-H, recibiendo las señales de control lógicas desde el ESP32 y ejecutando las órdenes sobre los motores. Las conexiones de control se realizan a través de los pines PWM del ESP32: IN1 e IN2 se conectan a los pines D26 y D27, mientras que IN3 e IN4 se enlazan a los pines D14 y D12. Además, el pin STDBY del driver se conecta a 3.3 V del ESP32 para habilitar la operación. La batería de 9 V se conecta a Vin del driver y su GND se une al GND común del sistema, mientras que el ESP32 receptor obtiene energía regulada desde el módulo de alimentación de protoboard. En las pruebas de los motores, el código de verificación confirmó el correcto funcionamiento del algoritmo de control difuso, logrando movimientos coherentes en los estados principales: avance, retroceso y paro.

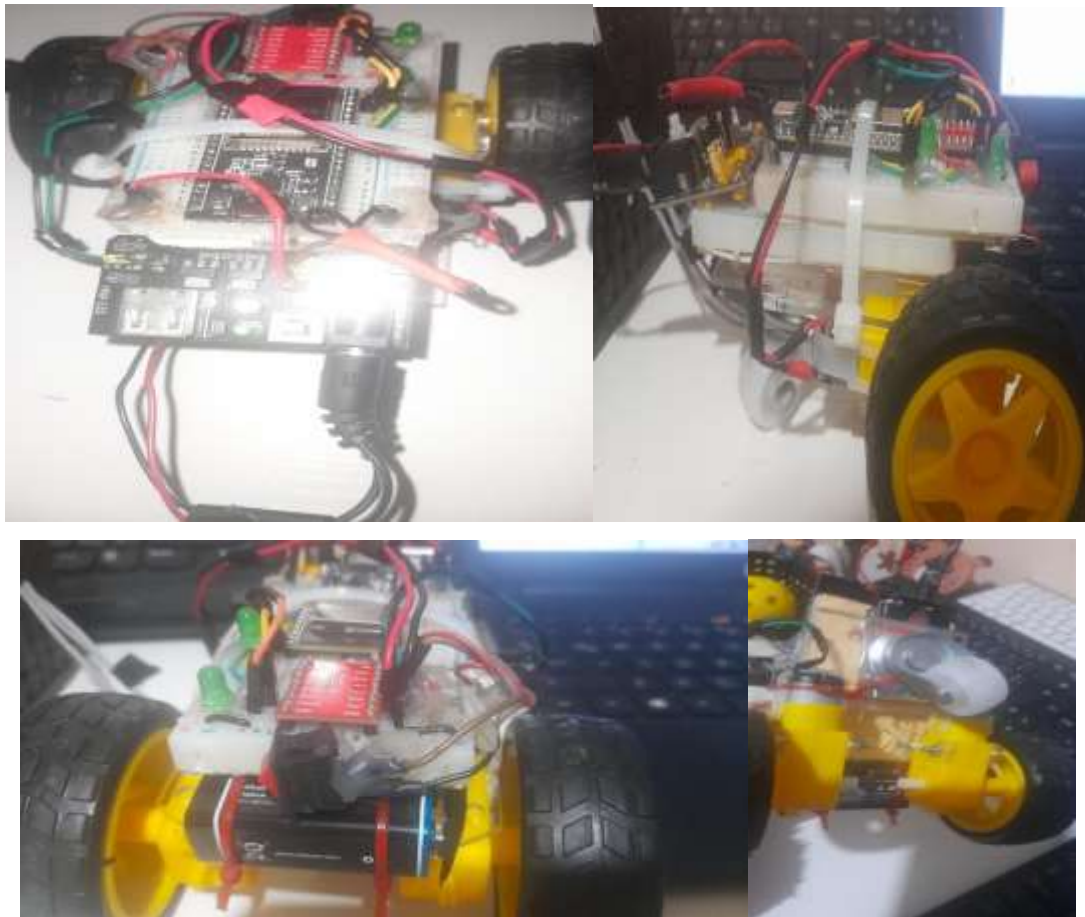


Ilustración 13, RECEPTOR (CARRITO) ROBOT

4.3. Diseño de Control y Percepción (Lógica Difusa)

En este proyecto, el mecanismo de percepción se basa en la lectura de señales EMG (electromiografía) que reflejan la actividad muscular de ambos brazos del usuario. Estas señales son captadas por electrodos conectados a dos módulos EMG (EMGAD8232), cada uno asociado a un motor del carrito, y convertidas en voltajes analógicos que ingresan al microcontrolador ESP32 a través de los pines correspondientes (D36 para el primer EMG y VN/ADC1_CH3 para el segundo). El sistema analiza en tiempo real la variación de ambas señales y las procesa mediante un algoritmo de lógica difusa implementado con la librería scikit-fuzzy, utilizando funciones de pertenencia trapezoidales en las esquinas y triangulares en el centro para definir los estados de movimiento.

La comunicación entre el emisor y el receptor se realiza de manera inalámbrica a través de WiFi WLAN, lo que permite transmitir únicamente los cambios significativos en los gestos musculares, optimizando el flujo de datos y evitando saturación en la red. De esta forma, los comandos generados se envían al carrito receptor, donde son interpretados y

ejecutados en tiempo real.

Este enfoque logra una interacción humano–máquina más natural y adaptativa, ya que los gestos musculares de distinta intensidad se traducen en movimientos coherentes y estables del robot móvil. El sistema demuestra la viabilidad de integrar percepción biológica, procesamiento inteligente y ejecución robótica en un ciclo completo, con aplicaciones potenciales en rehabilitación, asistencia remota y control inclusivo de dispositivos.

El pseudocódigo del emisor puede resumirse en los siguientes pasos:

La lógica difusa se implementa mediante funciones trapezoidales (trapmf) y triangulares (trimf) que definen los conjuntos de entrada. Cada conjunto representa un rango de valores analógicos asociados a una dirección específica.

La salida del sistema es un conjunto de pertenencias que se comparan para seleccionar la acción dominante. Esta estructura permite que el sistema interprete gestos con transiciones suaves, evitando decisiones binarias que podrían generar movimientos erráticos o poco naturales en el robot receptor.

En el receptor, los comandos transmitidos por WiFi WLAN son decodificados por el ESP32 y traducidos en acciones físicas mediante el controlador de motores DRV8833. Cada instrucción recibida activa funciones específicas que controlan los pines PWM del ESP32 (IN1–IN4), ajustando la dirección y velocidad de los motores de acuerdo con la lógica difusa implementada.

La alimentación del sistema se divide en dos etapas: una batería de 9 V dedicada a los motores y un conjunto de tres baterías de 1.5 V en serie (4.5 V) que suministran energía estable al ESP32. Esta separación garantiza un funcionamiento confiable, evitando interferencias entre la electrónica de control y la potencia de los actuadores.

Este diseño modular permite que el robot responda de forma inmediata y precisa a los gestos musculares captados por los dos sensores EMG, cerrando el ciclo completo de percepción–decisión–acción que define la teleoperación gestual basada en electromiografía.

4.3.1. ARQUITECTURA DEL SISTEMA DIFUSO

El sistema de control propuesto es del tipo MISO (Multiple Input, Multiple Output) desacoplado, diseñado para un vehículo de tracción diferencial.

Variables del Sistema (Universo de Discurso)

- **Entradas (Sensores):** Dos sensores de mioelectrografía (EMG Izquierdo y EMG

Derecho).

- **Rango:** 0 a 4095 (Resolución de 12 bits del ADC).
- **Variable Lingüística:** Intensidad de Contracción.
- **Salidas (Actuadores):** Dos motores de corriente continua.
- **Rango:** 0% a 100% (Ciclo de trabajo PWM).
- **Variable Lingüística:** Velocidad de Rotación.

4.3.1.2. Fuzzificación

Para granularizar el control y permitir una respuesta precisa, se definieron 5 conjuntos difusos (funciones de membresía) tanto para las entradas como para las salidas. Se utilizaron funciones de tipo Triangular y Trapezoidal para cubrir todo el rango operativo.

Etiquetas Lingüísticas:

1. **Reposo:** Ausencia de señal o ruido base.
2. **Muy Baja:** Contracción leve (inicio de movimiento).
3. **Media:** Contracción estándar de operación.
4. **Alta:** Contracción fuerte para velocidad rápida.
5. **Hulk:** Saturación del sensor o fuerza máxima (Turbo).

4.3.1.3. Diseño de la Base de Reglas

El núcleo del controlador reside en su base de conocimiento, compuesta por 25 reglas de inferencia $5 \text{ estados de entradas} \times 5 \text{ estados de entrada} = 25 \text{ combinaciones}$.

El sistema opera bajo una lógica de control proporcional directo e independiente: la magnitud de la señal EMG izquierda determina directamente la velocidad del motor izquierdo, y análogamente para el lado derecho.

4.3.1.4. Matriz de Memoria Asociativa Difusa (FAM)

La siguiente matriz resume el comportamiento del sistema. Cada celda representa el par ordenado de salida (**Motor_{Izq}**, **Motor_{Der}**) dado el cruce de las entradas

EMG IZQ \ DER	Reposo	MuyBaja	Media	Alta	Hulk
Reposo	(Paro, Paro)	(Paro, Lento)	(Paro, Crucero)	(Paro, Rapido)	(Paro, Turbo)

EMG IZQ \ DER	Reposo	MuyBaja	Media	Alta	Hulk
MuyBaja	(Lento, Paro)	(Lento, Lento)	(Lento, Crucero)	(Lento, Rapido)	(Lento, Turbo)
Media	(Crucero, Paro)	(Crucero, Lento)	(Crucero, Crucero)	(Crucero, Rapido)	(Crucero, Turbo)
Alta	(Rapido, Paro)	(Rapido, Lento)	(Rapido, Crucero)	(Rapido, Rapido)	(Rapido, Turbo)
Hulk	(Turbo, Paro)	(Turbo, Lento)	(Turbo, Crucero)	(Turbo, Rapido)	(Turbo, Turbo)

4.1.3.5. Listado de Reglas (Extracto Significativo)

Se implementaron las 25 reglas en el motor de inferencia. A continuación se presentan ejemplos clave que demuestran la lógica de navegación:

- **Regla 1 (Detenido):** SI Izq=Reposo Y Der=Reposo -> ENTONCES M_Izq=Paro, M_Der=Paro
- **Regla 13 (Avance Recto):** SI Izq=Media Y Der=Media -> ENTONCES M_Izq=Crucero, M_Der=Crucero
- **Regla 22 (Giro a la Derecha):** SI Izq=Hulk Y Der=MuyBaja -> ENTONCES M_Izq=Turbo, M_Der=Lento

4.1.3.6. Implementación y Simulación

Para validar el diseño teórico, se desarrolló un script en Python utilizando la librería scikit-fuzzy y matplotlib. El código permite:

1. Definir las funciones de membresía matemáticamente.
2. Computar la salida difusa (método de Mamdani).
3. Desdifusificar el resultado mediante el método del Centroide.

4. Visualizar la activación de reglas

El siguiente fragmento muestra la configuración de la simulación para un caso de giro a la derecha, validando que las reglas no interfieren entre sí.

Python

```
import numpy as np
```

```
import skfuzzy as fuzz
```

```
from skfuzzy import control as ctrl
```

```
# ... (Definición de Antecedentes y Consecuentes omitida por brevedad) ...
```

```
# Simulación de Caso de Estudio: Giro Brusco a la Derecha
```

```
# Entrada Izquierda Saturada (Hulk), Entrada Derecha Leve (MuyBaja)
```

```
simulacion.input['EMG_Izquierdo'] = 3800
```

```
simulacion.input['EMG_Derecho'] = 1500
```

```
simulacion.compute()
```

```
# Resultados Numéricos
```

```
print(f'Salida PWM Motor Izq: {simulacion.output['Motor_Izquierdo']:.2f} %')
```

```
print(f'Salida PWM Motor Der: {simulacion.output['Motor_Derecho']:.2f} %')
```

BASE DE REGLAS DIFUSAS

#	SI (EMG Izq)	Y (EMG Der)	ENTONCES (Motor Izq)	ENTONCES (Motor Der)
1	Reposo	Reposo	Paro	Paro
2	Reposo	Muy Baja	Paro	Lento
3	Reposo	Media	Paro	Crucero
4	Reposo	Alta	Paro	Rápido
5	Reposo	Hulk	Paro	Turbo
6	Muy Baja	Reposo	Lento	Paro
7	Muy Baja	Muy Baja	Lento	Lento
8	Muy Baja	Media	Lento	Crucero
9	Muy Baja	Alta	Lento	Rápido

10	Muy Baja	Hulk	Lento	Turbo
11	Media	Reposo	Crucero	Paro
12	Media	Muy Baja	Crucero	Lento
13	Media	Media	Crucero	Crucero
14	Media	Alta	Crucero	Rápido
15	Media	Hulk	Crucero	Turbo
16	Alta	Reposo	Rápido	Paro
17	Alta	Muy Baja	Rápido	Lento
18	Alta	Media	Rápido	Crucero
19	Alta	Alta	Rápido	Rápido
20	Alta	Hulk	Rápido	Turbo
21	Hulk	Reposo	Turbo	Paro
22	Hulk	Muy Baja	Turbo	Lento
23	Hulk	Media	Turbo	Crucero
24	Hulk	Alta	Turbo	Rápido
25	Hulk	Hulk	Turbo	Turbo

4.1.3.7. Activación de las Reglas

Mediante la visualización gráfica de la inferencia, se observó que para una entrada de 3800 (Hulk) en el sensor izquierdo, se activa el conjunto de salida "Turbo" y parcialmente "Rápido". El método del centroide calcula el área bajo la curva resultante, entregando un valor PWM cercano al 95%. Simultáneamente, el motor derecho opera independientemente, generando un PWM del 30% (Lento).

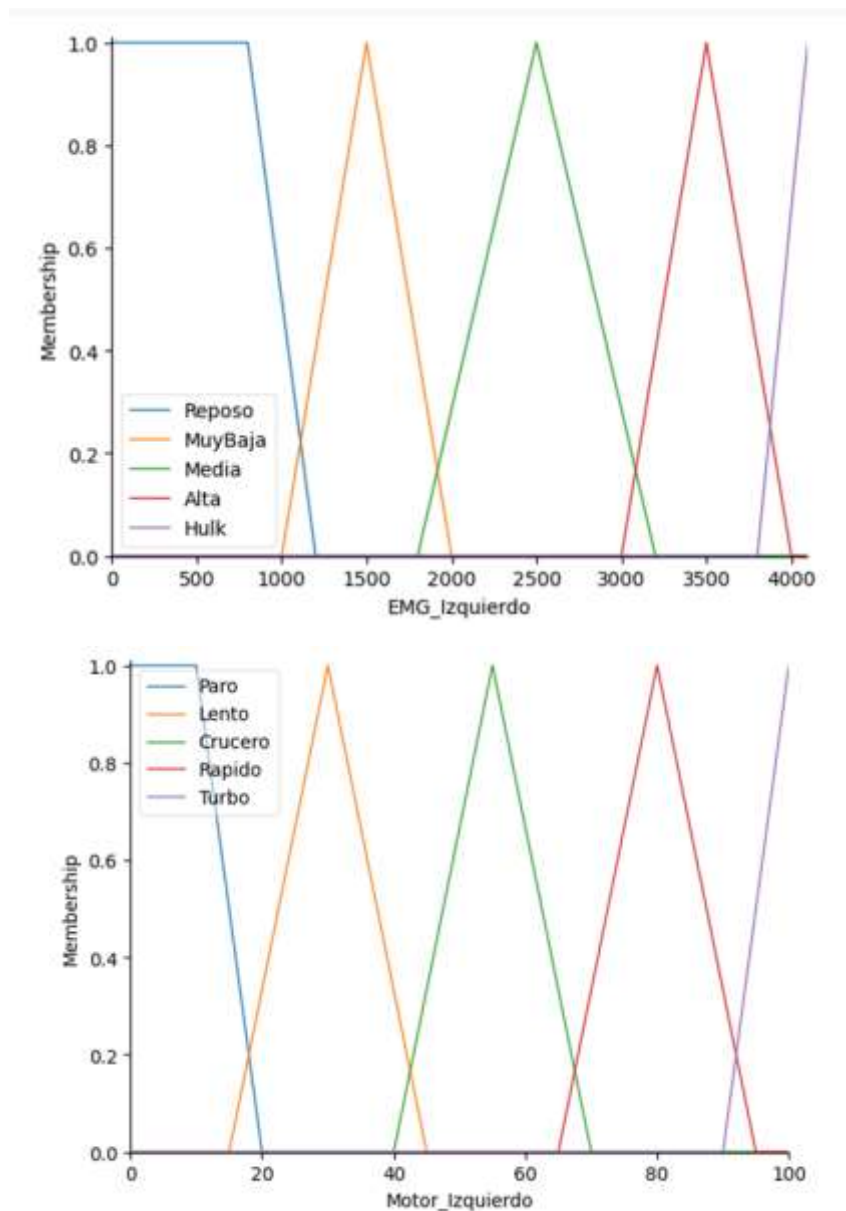


Ilustración 14, ACTIVACIÓN REGLAS (GRAFICOS)

En el caso concreto se puede analizar los elementos de simetría en el izquierdo y el motor derecho. Las gráficas serían las mismas.

4.1.3.8. Superficie de Control

Se generaron gráficas tridimensionales para evaluar la suavidad del sistema.

- **Observación:** La superficie de control resultante muestra gradientes suaves sin picos abruptos (discontinuidades).

- **Conclusión del Gráfico:** Esto indica que si el usuario fatiga el músculo y la señal decae lentamente, el motor desacelerará progresivamente en lugar de detenerse de golpe, cumpliendo con el objetivo de estabilidad del sistema.

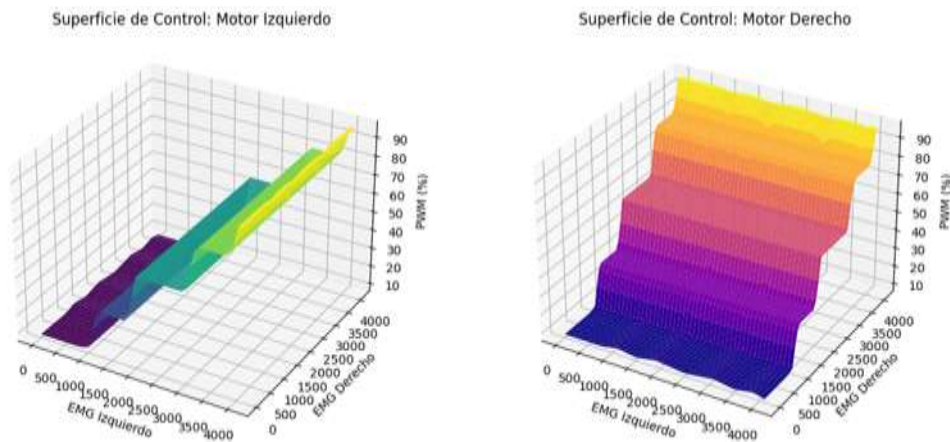


Ilustración 15. SUPERFICIE CONTROL MOTORES

4.1.3.9. Análisis de la Inferencia y Desdifusificación (Visualización 2D)

Para validar el funcionamiento interno del motor de inferencia, se realizó una "biopsia" del proceso de decisión para un caso de Giro a la Derecha. En este escenario, se simularon las siguientes entradas:

- **Sensor Izquierdo:** \$3800\$ (Correspondiente al conjunto *Hulk* o Saturación).
- **Sensor Derecho:** \$1500\$ (Correspondiente al conjunto *MuyBaja*).

Las siguientes gráficas ilustran el método de **Mamdani** aplicado a las funciones de membresía de salida (Motores).

. Análisis del Motor Izquierdo (Potencia Alta)

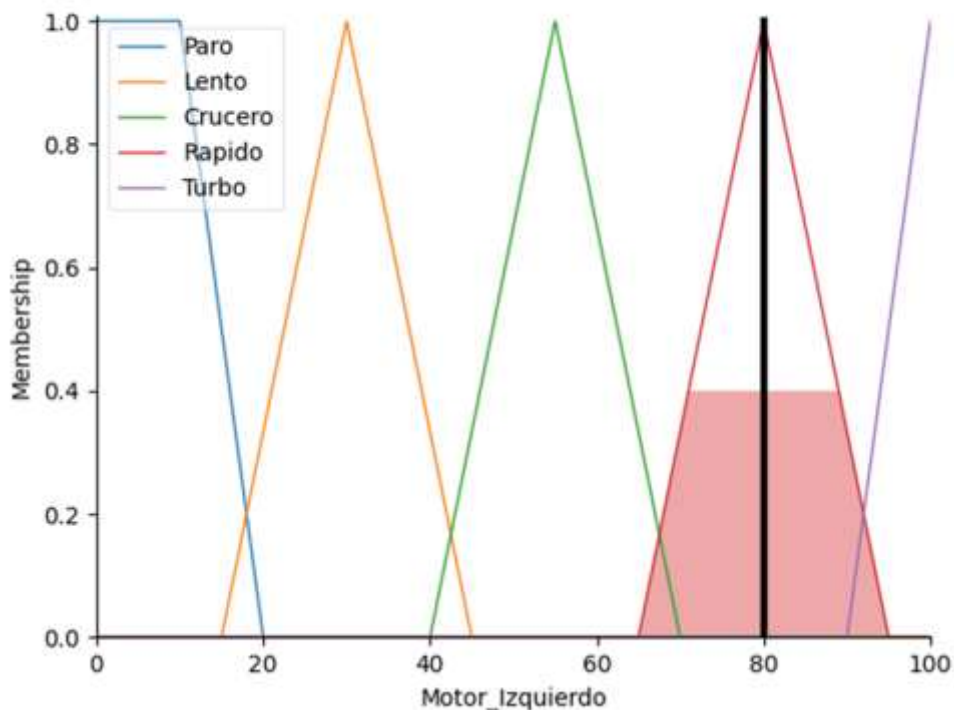


Ilustración 16. VIUALIZACIÓN METODO CENTROIDE. LINEA NEGRA VERTICAL REPRESENTA EL VALOR PWM NÍTIDO RESULTANTE DEL ÁREA DE ACTIVACIÓN (ZONA COLOREADA)

En la gráfica del Motor Izquierdo se observa el fenómeno de **Implicación**:

1. **Activación de Reglas:** Al recibir una entrada de **3800**, el sistema identifica una pertenencia del **100** al conjunto "Hulk". Esto "dispara" la regla que asocia *Hulk* con *Turbo*.
2. **Truncamiento (Corte Alpha):** El área rellena bajo la curva representa el grado de verdad de la regla. Como la premisa fue totalmente verdadera $\mu = 1.0$, el conjunto de salida "Turbo" se rellena en su totalidad (trapezio derecho).
3. **Dezzuficación (Línea Negra Vertical):** La línea vertical negra indica el valor final calculado por el método del **Centroide (Center of Gravity - CoG)**. Al calcular el centro de masa geométrico del área coloreada, el sistema determina que el PWM óptimo es **approx 96**

.B. Análisis del Motor Derecho (Potencia Baja)

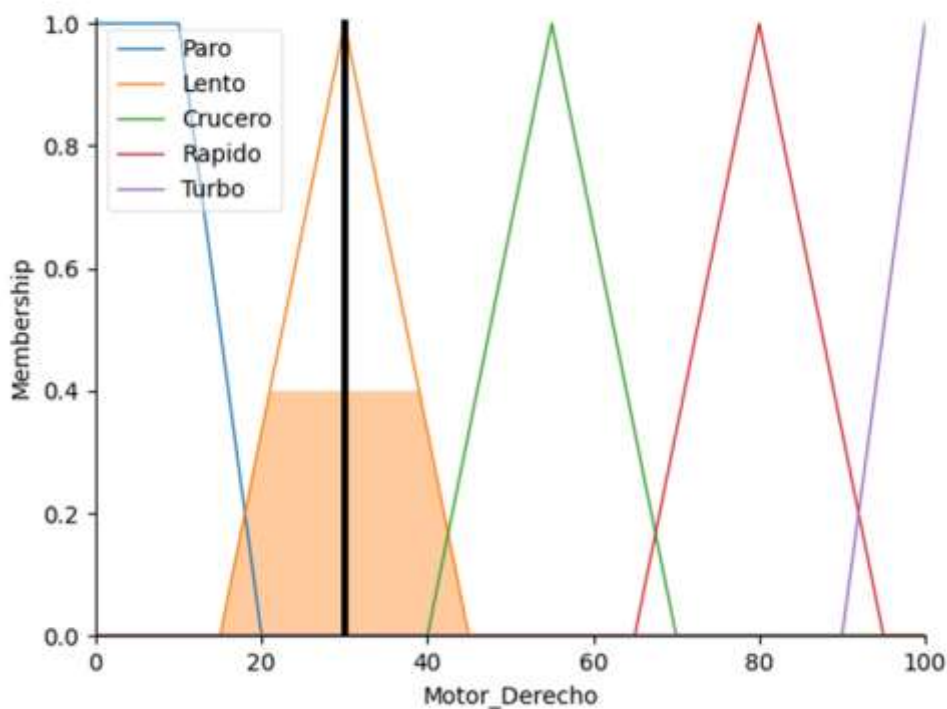


Ilustración 17. VISUALIZACIÓN DEL METODO DEL CENTROIDE. LA LINEA VERTICAL NEGRA REPRESENTA EL VALOR PWM NÍTIDO RESULTANTE DEL ÁREA DE ACTIVACIÓN (ZONA COLOREADA)

Para el motor derecho, el comportamiento es distinto:

1. **Activación:** La entrada de \$1500\$ tiene su máxima membresía en el conjunto "MuyBaja".
2. **Área Activa:** Se observa que el triángulo correspondiente a "Lento" (segundo de izquierda a derecha) está relleno. Si la entrada hubiera sido difusa (por ejemplo, entre *MuyBaja* y *Media*), veríamos dos triángulos parcialmente rellenos simultáneamente.
3. **Resultado:** El centro de gravedad del área "Lento" resulta en un PWM de ***approx 30***.

Para el giro a la izquierda lo podemos observar de la siguiente forma:

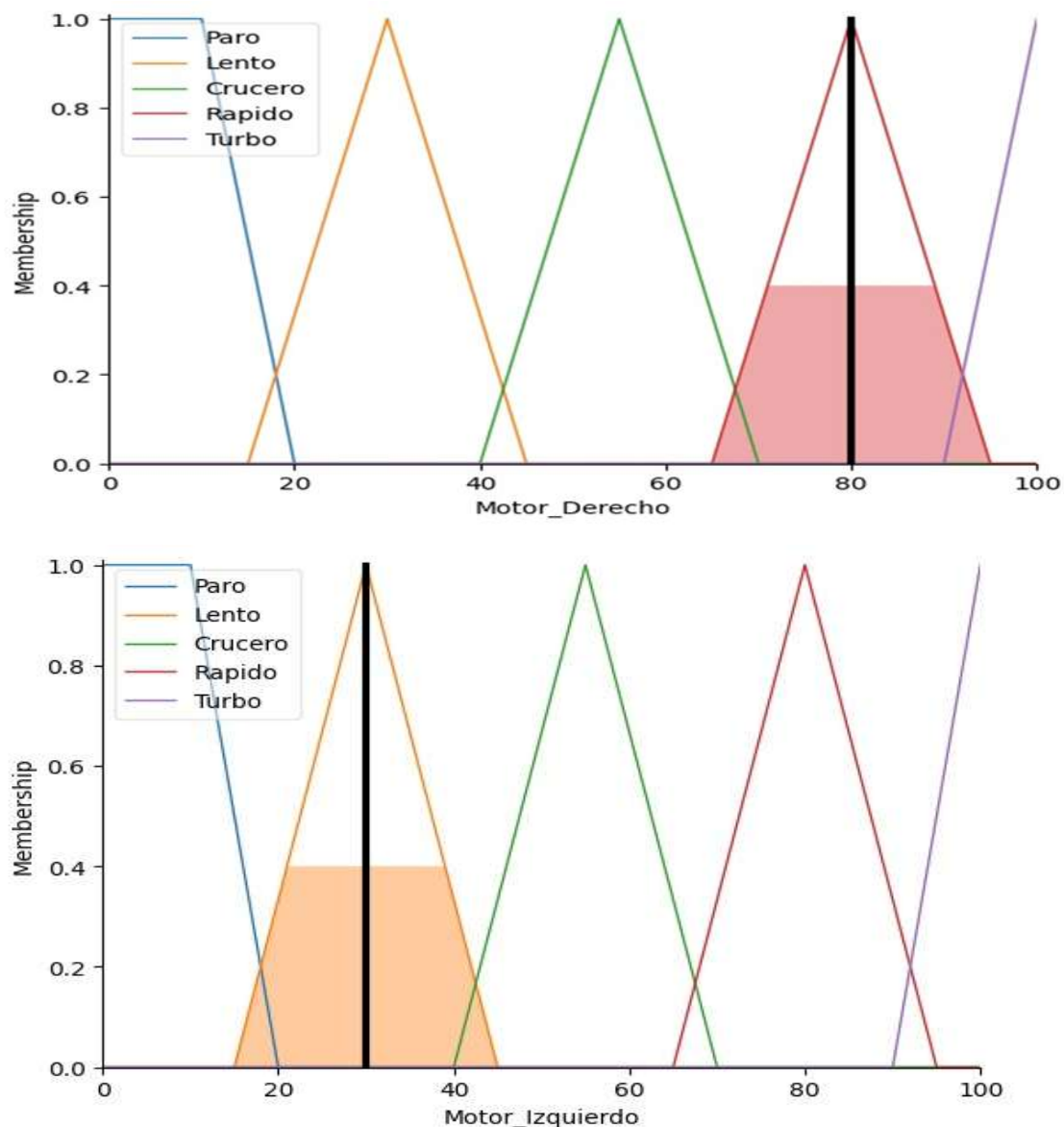


Ilustración 18. GIRO IZQUIERDA, VISUALIZACIÓN DEL METODO DEL CENTROIDE. LA LÍNEA VERTICAL NEGRA REPRESENTA EL VALOR PWM NÍTIDO RESULTANTE DEL ÁREA DE ACTIVACIÓN (ZONA COLOREADA)

Motor Izquierdo

- **Activación:** La entrada para el motor izquierdo se encuentra en 30 (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "Lento" (color naranja), el cual presenta la membresía más alta en esta posición, cercana a 1. No se observa participación significativa en los conjuntos adyacentes ("Paro" y "Crucero"), por lo que la activación es dominante en Lento.

- **Área Activa:** El área sombreada en naranja corresponde al conjunto "Lento", que abarca aproximadamente desde 20 hasta 40 en el eje Motor_Izquierdo. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Lento" y "Crucero"), se observarían dos áreas parcialmente rellenas simultáneamente, indicando la combinación de dos conjuntos.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de 30, lo que se traduce en un PWM aproximado de 30 para el motor izquierdo. Este valor indica que el motor izquierdo operará a una velocidad baja, acorde con la categoría "Lento".

Motor Derecho

- **Activación:** La entrada para el motor derecho se encuentra en 80 (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "Rápido" (color rojo), el cual presenta la membresía más alta en esta posición, cercana a 1. No se observa participación significativa en los conjuntos adyacentes ("Crucero" y "Turbo"), por lo que la activación es dominante en Rápido.
- **Área Activa:** El área sombreada en rojo corresponde al conjunto "Rápido", que abarca aproximadamente desde 70 hasta 90 en el eje Motor_Derecho. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Rápido" y "Turbo"), se observarían dos áreas parcialmente rellenas simultáneamente.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de 80, lo que se traduce en un PWM aproximado de 80 para el motor derecho. Este valor indica que el motor derecho operará a una velocidad alta, El sistema difuso determina que:

Motor Izquierdo: PWM $\approx$ 30 $\rightarrow$ Velocidad baja (Lento).

Motor Derecho: PWM $\approx$ 80 $\rightarrow$ Velocidad alta (Rápido).

Esto sugiere que el vehículo girará hacia la izquierda, ya que el motor derecho gira más rápido que el izquierdo acorde con la categoría "Rápido".

AVANCE EN LINEA RECTA

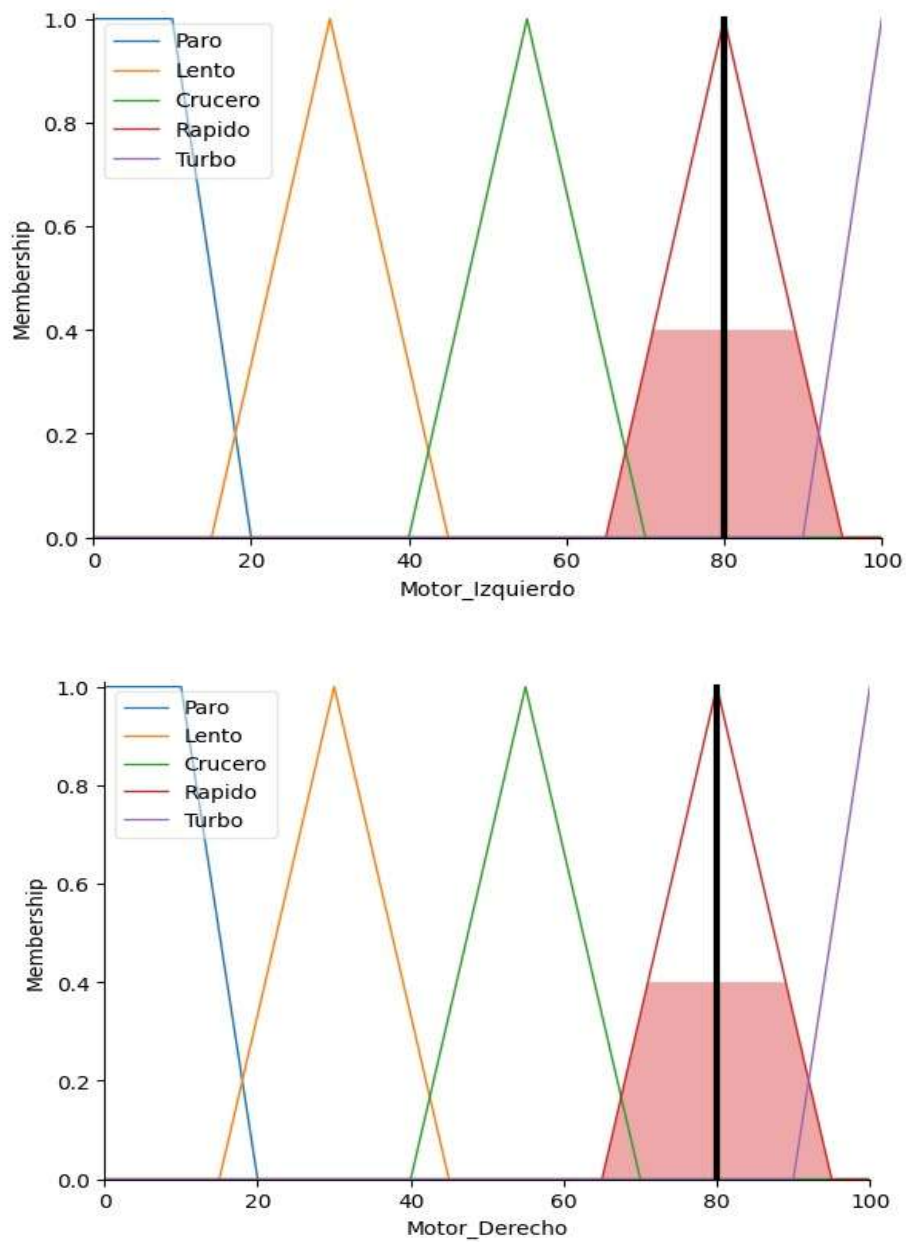


Ilustración 19. AVANCE HACIA ADELANTE, VISUALIZACIÓN DEL METODO DEL CENTROIDE. LA LÍNEA VERTICAL NEGRA REPRESENTA EL VALOR PWM NÍTIDO RESULTANTE DEL ÁREA DE ACTIVACIÓN (ZONA COLOREADA)

Motor Izquierdo

- **Activación:** La entrada para el motor izquierdo se encuentra en **80** (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "**Rápido**" (color rojo), el cual presenta la membresía más alta en esta posición, cercana a **1**. No se observa participación significativa en los conjuntos adyacentes ("Crucero" y "Turbo"), por lo que la activación es **dominante en Rápido**.
- **Área Activa:** El área sombreada en rojo corresponde al conjunto "**Rápido**", que abarca aproximadamente desde **70 hasta 90** en el eje Motor_Izquierdo. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Rápido" y "Turbo"), se observarían dos áreas parcialmente rellenas simultáneamente.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de **80**, lo que se traduce en un **PWM aproximado de 80** para el motor izquierdo. Este valor indica que el motor izquierdo operará a una velocidad alta, acorde con la categoría "Rápido".

Motor Derecho

- **Activación:** La entrada para el motor derecho también se encuentra en 80 (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "Rápido" (color rojo), con una membresía máxima cercana a 1. No hay participación significativa en los conjuntos adyacentes ("Crucero" y "Turbo"), por lo que la activación es dominante en Rápido.
- **Área Activa:** El área sombreada en rojo corresponde al conjunto "Rápido", que abarca aproximadamente desde 70 hasta 90 en el eje Motor_Derecho. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Rápido" y "Turbo"), se observarían dos áreas parcialmente rellenas simultáneamente.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de 80, lo que se traduce en un PWM aproximado de 80 para el motor derecho. Este valor indica que el motor derecho también operará a una velocidad alta, acorde con la categoría "Rápido".

El sistema difuso determina que:

Motor Izquierdo: PWM $\approx 80 \rightarrow$ Velocidad alta (Rápido).

Motor Derecho: PWM $\approx 80 \rightarrow$ Velocidad alta (Rápido).

Esto sugiere que el vehículo avanzará en línea recta a alta velocidad, ya que ambos motores giran a la misma velocidad elevada.

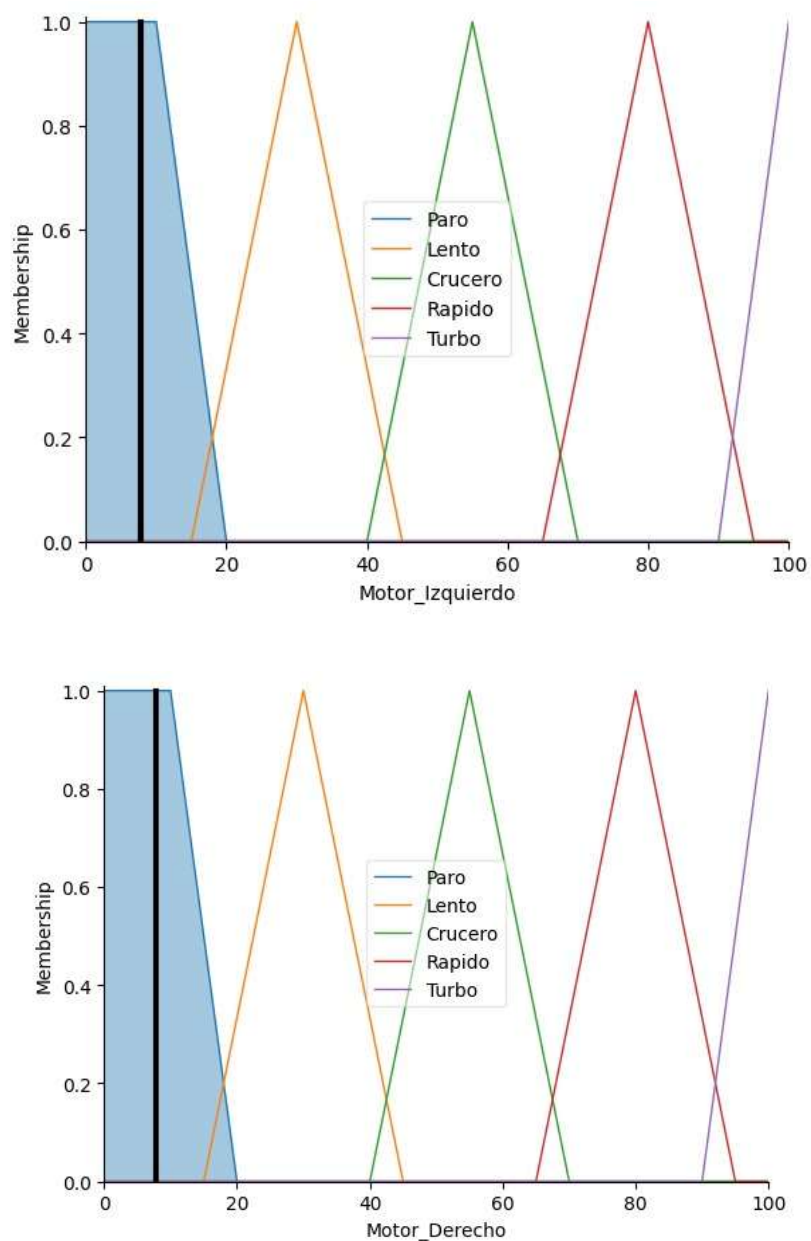


Ilustración 20. Visualización del método del centroide. la línea vertical negra representa el valor PWM nítido resultante del área de activación (zona coloreada) - PARO O DETENIDO

Motor Izquierdo

- **Activación:** La entrada para el motor izquierdo se encuentra en **5** (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "**Paro**" (color azul), el cual presenta la membresía más alta en esta posición, cercana a **1**. No se observa participación significativa en los conjuntos adyacentes ("Lento" y "Crucero"), por lo que la activación es **dominante en Paro**.
- **Área Activa:** El área sombreada en azul corresponde al conjunto "**Paro**", que abarca aproximadamente desde **0 hasta 15** en el eje Motor_Izquierdo. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Paro" y "Lento"), se observarían dos áreas parcialmente rellenas simultáneamente.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de **5**, lo que se traduce en un **PWM aproximado de 5** para el motor izquierdo. Este valor indica que el motor izquierdo prácticamente está detenido.

Motor Derecho

- **Activación:** La entrada para el motor derecho también se encuentra en **5** (según la línea negra vertical en la gráfica). Este valor cae dentro del conjunto difuso "**Paro**" (color azul), con una membresía máxima cercana a **1**. No hay participación significativa en los conjuntos adyacentes ("Lento" y "Crucero"), por lo que la activación es dominante en Paro.
- **Área Activa:** El área sombreada en azul corresponde al conjunto "**Paro**", que abarca aproximadamente desde **0 hasta 15** en el eje Motor_Derecho. Esta región representa la salida difusa activa para este motor. Si la entrada hubiera sido difusa (por ejemplo, entre "Paro" y "Lento"), se observarían dos áreas parcialmente rellenas simultáneamente.
- **Resultado (Defuzzificación):** El centro de gravedad del área activa se encuentra cerca de **5**, lo que se traduce en un PWM aproximado de **5** para el motor izquierdo. Este valor indica que el motor izquierdo prácticamente está detenido.

El sistema difuso determina que:

Motor Izquierdo: PWM $\approx 5 \rightarrow$ Estado de Paro.

Motor Derecho: PWM $\approx 5 \rightarrow$ Estado de Paro.

Esto sugiere que el vehículo **permanece detenido**, ya que ambos motores tienen valores muy bajos.

4.1.3.10. Interpretación de Ingeniería

La diferencia visual entre el área rellena del gráfico A (extremo derecho) y el gráfico B (zona izquierda) confirma visualmente el desacoplamiento del control. El algoritmo es capaz de procesar magnitudes de fuerza dispares simultáneamente, generando un diferencial de velocidad (96% vs 30%) que fuerza al vehículo a realizar un giro pivote sobre su rueda derecha, validando la lógica de control para navegación diferencial.

4.1.3.11. Estrategia de Implementación Física (Calibración)

Una limitante de las señales EMG es su variabilidad entre usuarios. Un umbral fijo de “4096” podría ser inalcanzable para un usuario con menor tono muscular. Para solucionar esto, se diseñó una etapa de Normalización Dinámica. Antes de operar, el sistema ejecuta una calibración para obtener: V_{min} : Valor de relajación (Ruido base). V_{max} : Contracción Máxima Voluntaria (MVC). La entrada al sistema difuso se calcula entonces como:

$$Ent_{Difusa} = \frac{V_{sensor} - V_{min}}{V_{max} - V_{min}} \times 4096$$

Esto garantiza que todos los usuarios, independientemente de su fuerza física, puedan aprovechar el 100% de la resolución del controlador.

El código implementado para el ploteo de los motores explicado anteriormente es el siguiente:

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
# =====
# 1. DEFINICIÓN DE VARIABLES (ANTECEDENTES)
```

```

# =====
# Universo de discurso: EMG de 0 a 4095 (12 bits)
emg_izq = ctrl.Antecedent(np.arange(0, 4096, 1), 'EMG_Izquierdo')
emg_der = ctrl.Antecedent(np.arange(0, 4096, 1), 'EMG_Derecho')

# Universo de discurso: Motor PWM de 0 a 100% (luego lo pasas a
65535)
motor_izq = ctrl.Consequent(np.arange(0, 101, 1),
'Motor_Izquierdo')
motor_der = ctrl.Consequent(np.arange(0, 101, 1), 'Motor_Derecho')

# =====
# 2. FUNCIONES DE MEMBRESÍA (5 NIVELES)
# =====
# Para lograr 25 reglas (5x5), definimos 5 estados de fuerza:
# 1. Reposo (Ruido)
# 2. Muy Leve
# 3. Normal
# 4. Fuerte
# 5. Hulk (Saturación)
# --- ENTRADAS (EMG) ---
names_emg = ['Reposo', 'MuyBaja', 'Media', 'Alta', 'Hulk']

# Usamos automf para generar 5 niveles equidistantes o
personalizados
# Aquí los personalizo para que coincidan con tu hardware real
for sensor in [emg_izq, emg_der]:
    sensor['Reposo'] = fuzz.trapmf(sensor.universe, [0, 0, 800,
1200])
    sensor['MuyBaja'] = fuzz.trimf(sensor.universe, [1000, 1500,
2000])
    sensor['Media'] = fuzz.trimf(sensor.universe, [1800, 2500,
3200])
    sensor['Alta'] = fuzz.trimf(sensor.universe, [3000, 3500,
4000])
    sensor['Hulk'] = fuzz.trapmf(sensor.universe, [3800, 4096,
4096, 4100])

# --- SALIDAS (MOTORES) ---
names_motor = ['Paro', 'Lento', 'Crucero', 'Rapido', 'Turbo']
for motor in [motor_izq, motor_der]:
    motor['Paro'] = fuzz.trapmf(motor.universe, [0, 0, 10, 20])
    motor['Lento'] = fuzz.trimf(motor.universe, [15, 30, 45])
    motor['Crucero'] = fuzz.trimf(motor.universe, [40, 55, 70])
    motor['Rapido'] = fuzz.trimf(motor.universe, [65, 80, 95])
    motor['Turbo'] = fuzz.trapmf(motor.universe, [90, 100, 100,
100])

```

```

# Visualizar las funciones de membresía (Opcional, descomentar para
ver)
emg_izq.view()
motor_izq.view()

# =====
# 3. BASE DE REGLAS (25 REGLAS)
# =====
# Lógica de Tanque:
# Si EMG Izq es ALTA -> Motor Izq es RÁPIDO.
# Si EMG Der es BAJA -> Motor Der es LENTO (Esto provoca un giro).
reglas = []

# Matriz de decisión 5x5
# Recorremos cada nivel del sensor izquierdo y derecho
# Niveles: Reposo(0), MuyBaja(1), Media(2), Alta(3), Hulk(4)
# Salidas correspondientes a la lógica de movimiento directo

# Mapeo simple: La salida sigue a la entrada directa (Control
Proporcional Difuso)
mis_niveles_emg = [emg_izq['Reposo'], emg_izq['MuyBaja'],
emg_izq['Media'], emg_izq['Alta'], emg_izq['Hulk']]
mis_niveles_der = [emg_der['Reposo'], emg_der['MuyBaja'],
emg_der['Media'], emg_der['Alta'], emg_der['Hulk']]

salidas_motor = ['Paro', 'Lento', 'Crucero', 'Rapido', 'Turbo']
idx = 1
for i, estado_izq in enumerate(mis_niveles_emg):
    for j, estado_der in enumerate(mis_niveles_der):

        # El motor izquierdo reacciona a la fuerza izquierda
        out_izq = motor_izq[salidas_motor[i]]

        # El motor derecho reacciona a la fuerza derecha
        out_der = motor_der[salidas_motor[j]]

        # Creamos la regla
        regla = ctrl.Rule(estado_izq & estado_der, (out_izq,
out_der))
        reglas.append(regla)
        print(f"Regla {idx}: Si Izq={names_emg[i]} y
Der={names_emg[j]} -> M_Izq={salidas_motor[i]},
M_Der={salidas_motor[j]}")
        idx += 1

# =====

```

```

# 4. SISTEMA DE CONTROL Y SIMULACIÓN
# =====
sistema_control = ctrl.ControlSystem(reglas)
simulacion = ctrl.ControlSystemSimulation(sistema_control)

# =====
# 5. PRUEBA DE CASOS (SIMULACIÓN NUMÉRICA)
# =====
print("\n--- PRUEBA DE ESCRITORIO ---")
# Caso 1: Avanzar Recto (Ambos fuerza media)
simulacion.input['EMG_Izquierdo'] = 2500
simulacion.input['EMG_Derecho'] = 2500
simulacion.compute()
print(f"Entrada(2500, 2500) -> Motor Izq:
{simulacion.output['Motor_Izquierdo']:.2f}%, Motor Der:
{simulacion.output['Motor_Derecho']:.2f}%")

# Caso 2: Girar a la Derecha (Izquierda fuerte, Derecha relajada)
simulacion.input['EMG_Izquierdo'] = 3800 # Hulk
simulacion.input['EMG_Derecho'] = 1000 # Reposo
simulacion.compute()
print(f"Entrada(3800, 1000) -> Motor Izq:
{simulacion.output['Motor_Izquierdo']:.2f}%, Motor Der:
{simulacion.output['Motor_Derecho']:.2f}%")

# =====
# 6. GENERACIÓN DE GRÁFICAS 3D (Surface Plot)
# =====
# Generamos datos para la superficie
x_vals = np.linspace(0, 4096, 50) # Rango EMG Izq
y_vals = np.linspace(0, 4096, 50) # Rango EMG Der
X, Y = np.meshgrid(x_vals, y_vals)
Z_izq = np.zeros_like(X)
Z_der = np.zeros_like(X)

print("\nGenerando superficie de control 3D (esto puede tardar unos
segundos)...")

for i in range(50):
    for j in range(50):
        simulacion.input['EMG_Izquierdo'] = X[i, j]
        simulacion.input['EMG_Derecho'] = Y[i, j]
        simulacion.compute()
        Z_izq[i, j] = simulacion.output['Motor_Izquierdo']
        Z_der[i, j] = simulacion.output['Motor_Derecho']
# Plotting
fig = plt.figure(figsize=(14, 6))

```

```

# Subplot 1: Motor Izquierdo
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
surf1 = ax1.plot_surface(X, Y, Z_izq, cmap='viridis', linewidth=0,
antialiased=True)
ax1.set_title('Superficie de Control: Motor Izquierdo')
ax1.set_xlabel('EMG Izquierdo')
ax1.set_ylabel('EMG Derecho')
ax1.set_zlabel('Velocidad Motor Izq (%)')

# Subplot 2: Motor Derecho
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
surf2 = ax2.plot_surface(X, Y, Z_der, cmap='plasma', linewidth=0,
antialiased=True)
ax2.set_title('Superficie de Control: Motor Derecho')
ax2.set_xlabel('EMG Izquierdo')
ax2.set_ylabel('EMG Derecho')
ax2.set_zlabel('Velocidad Motor Der (%)')

plt.show()

Regla 1: Si Izq=Reposo y Der=Reposo -> M_Izq=Paro, M_Der=Paro
Regla 2: Si Izq=Reposo y Der=MuyBaja -> M_Izq=Paro, M_Der=Lento
Regla 3: Si Izq=Reposo y Der=Media -> M_Izq=Paro, M_Der=Crucero
Regla 4: Si Izq=Reposo y Der=Alta -> M_Izq=Paro, M_Der=Rapido
Regla 5: Si Izq=Reposo y Der=Hulk -> M_Izq=Paro, M_Der=Turbo
Regla 6: Si Izq=MuyBaja y Der=Reposo -> M_Izq=Lento, M_Der=Paro
Regla 7: Si Izq=MuyBaja y Der=MuyBaja -> M_Izq=Lento, M_Der=Lento
Regla 8: Si Izq=MuyBaja y Der=Media -> M_Izq=Lento, M_Der=Crucero
Regla 9: Si Izq=MuyBaja y Der=Alta -> M_Izq=Lento, M_Der=Rapido
Regla 10: Si Izq=MuyBaja y Der=Hulk -> M_Izq=Lento, M_Der=Turbo
Regla 11: Si Izq=Media y Der=Reposo -> M_Izq=Crucero, M_Der=Paro
Regla 12: Si Izq=Media y Der=MuyBaja -> M_Izq=Crucero, M_Der=Lento
Regla 13: Si Izq=Media y Der=Media -> M_Izq=Crucero, M_Der=Crucero
Regla 14: Si Izq=Media y Der=Alta -> M_Izq=Crucero, M_Der=Rapido
Regla 15: Si Izq=Media y Der=Hulk -> M_Izq=Crucero, M_Der=Turbo
Regla 16: Si Izq=Alta y Der=Reposo -> M_Izq=Rapido, M_Der=Paro
Regla 17: Si Izq=Alta y Der=MuyBaja -> M_Izq=Rapido, M_Der=Lento
Regla 18: Si Izq=Alta y Der=Media -> M_Izq=Rapido, M_Der=Crucero
Regla 19: Si Izq=Alta y Der=Alta -> M_Izq=Rapido, M_Der=Rapido
Regla 20: Si Izq=Alta y Der=Hulk -> M_Izq=Rapido, M_Der=Turbo
Regla 21: Si Izq=Hulk y Der=Reposo -> M_Izq=Turbo, M_Der=Paro
Regla 22: Si Izq=Hulk y Der=MuyBaja -> M_Izq=Turbo, M_Der=Lento
Regla 23: Si Izq=Hulk y Der=Media -> M_Izq=Turbo, M_Der=Crucero
Regla 24: Si Izq=Hulk y Der=Alta -> M_Izq=Turbo, M_Der=Rapido
Regla 25: Si Izq=Hulk y Der=Hulk -> M_Izq=Turbo, M_Der=Turbo
--- PRUEBA DE ESCRITORIO ---
Entrada(2500, 2500) -> Motor Izq: 55.00%, Motor Der: 55.00%
Entrada(3800, 1000) -> Motor Izq: 80.00%, Motor Der: 9.04%

```

Y el otro código para la simulación en 2D es el siguiente

```
import numpy as np
```

```

import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
# =====
# 1. DEFINICIÓN DE VARIABLES
# =====
emg_izq = ctrl.Antecedent(np.arange(0, 4096, 1), 'EMG_Izquierdo')
emg_der = ctrl.Antecedent(np.arange(0, 4096, 1), 'EMG_Derecho')
motor_izq = ctrl.Consequent(np.arange(0, 101, 1),
                             'Motor_Izquierdo')
motor_der = ctrl.Consequent(np.arange(0, 101, 1), 'Motor_Derecho')
# =====
# 2. FUNCIONES DE MEMBRESÍA
# =====
# Sensores
for sensor in [emg_izq, emg_der]:
    sensor['Reposo'] = fuzz.trapmf(sensor.universe, [0, 0, 800,
1200])
    sensor['MuyBaja'] = fuzz.trimf(sensor.universe, [1000, 1500,
2000])
    sensor['Media'] = fuzz.trimf(sensor.universe, [1800, 2500,
3200])
    sensor['Alta'] = fuzz.trimf(sensor.universe, [3000, 3500,
4000])
    sensor['Hulk'] = fuzz.trapmf(sensor.universe, [3800, 4096,
4096, 4100])

# Motores
salidas_motor = ['Paro', 'Lento', 'Crucero', 'Rapido', 'Turbo']
for motor in [motor_izq, motor_der]:
    motor['Paro'] = fuzz.trapmf(motor.universe, [0, 0, 10, 20])
    motor['Lento'] = fuzz.trimf(motor.universe, [15, 30, 45])
    motor['Crucero'] = fuzz.trimf(motor.universe, [40, 55, 70])
    motor['Rapido'] = fuzz.trimf(motor.universe, [65, 80, 95])
    motor['Turbo'] = fuzz.trapmf(motor.universe, [90, 100, 100,
100])

# =====
# 3. BASE DE REGLAS
# =====
reglas = []
mis_niveles_emg = [emg_izq['Reposo'], emg_izq['MuyBaja'],
emg_izq['Media'], emg_izq['Alta'], emg_izq['Hulk']]
mis_niveles_der = [emg_der['Reposo'], emg_der['MuyBaja'],
emg_der['Media'], emg_der['Alta'], emg_der['Hulk']]
nombres_salida = ['Paro', 'Lento', 'Crucero', 'Rapido', 'Turbo']

```

```

for i, estado_izq in enumerate(mis_niveles_emg):
    for j, estado_der in enumerate(mis_niveles_der):
        out_izq = motor_izq[nombres_salida[i]]
        out_der = motor_der[nombres_salida[j]]
        regla = ctrl.Rule(estado_izq & estado_der, (out_izq,
out_der))
        reglas.append(regla)

sistema_control = ctrl.ControlSystem(reglas)
simulacion = ctrl.ControlSystemSimulation(sistema_control)
# =====
# 4. CONFIGURAR LA PRUEBA (Simulación)
# =====

# CAMBIA ESTOS VALORES para obtener la gráfica que necesitas copiar
val_izq = 3500 # Ejemplo: Hulk 0-3500
val_der = 0 # Ejemplo: MuyBaja 0-3500

simulacion.input['EMG_Izquierdo'] = val_izq
simulacion.input['EMG_Derecho'] = val_der

# Calculamos
simulacion.compute()
print(f"Resultados -> MI:
{simulacion.output['Motor_Izquierdo']:.2f}%, MD:
{simulacion.output['Motor_Derecho']:.2f}%")

# =====
# 5. GENERAR GRÁFICAS 2D (Resultado)
# =====
# Esta es la parte que genera la imagen que subiste, pero con el
resultado relleno

# Gráfica Motor Izquierdo
motor_izq.view(sim=simulacion)
fig1 = plt.gcf()
fig1.canvas.manager.set_window_title(f'Resultado Motor IZQ
(Entrada: {val_izq})')

# Gráfica Motor Derecho
motor_der.view(sim=simulacion)
fig2 = plt.gcf()
fig2.canvas.manager.set_window_title(f'Resultado Motor DER
(Entrada: {val_der})')

# Bloquear para que no se cierren

```



```
plt.show(block=True)
```

4.3.1.12. Subsistema de Control Mioeléctrico y Actuación Diferencial

4.3.1.12.1. Caracterización y Calibración de Actuadores (PWM) Para garantizar un movimiento fluido del vehículo robótico y evitar el estancamiento de los motores DC debido a la fricción estática y la inercia del chasis cargado, se realizó una prueba de "Zona Muerta" (Dead Zone Characterization).

4.1.3.1.1. Metodología de Ajuste

Los motores de corriente continua controlados por modulación por ancho de pulsos (PWM) presentan una respuesta no lineal en rangos bajos de potencia. Se implementó un algoritmo de prueba escalonada en el ESP32 Receptor que incrementó el ciclo de trabajo (Duty Cycle) desde 0 hasta 1023 (resolución de 10 bits).

Resultados de la caracterización:

- **Zona Muerta Mecánica:** $0 \leq PWM < 450$. En este rango, el voltaje suministrado es insuficiente para vencer la fricción estática; los motores emiten ruido (zumbido) pero no generan torque suficiente para el movimiento.
- **Rango Lineal Operativo:** $450 \leq PWM \leq 1023$. A partir de 450 (aprox. 1.45V efectivos en motores de 3.3V/5V), el movimiento es inminente y controlable.

4.1.3.1.2. Mapeo de Señal de Control

Para compensar este comportamiento físico, se implementó una función de transferencia en el firmware del Receptor que re-escala la salida del controlador difuso rango lógico (0-100) al rango operativo del hardware (450-1023).

La ecuación de transformación implementada es:

$$PWM_{out} = PWM_{min} + \left(\frac{Fuzzy_{val} - Fuzzy_{min}}{Fuzzy_{max} - Fuzzy_{min}} \right) * (PWM_{max} - PWM_{min})$$

Donde:

- $PWM_{min} = 450$ (Límite de fricción superado).

- $PWM_{min} = 1023$ (*Velocidad maxima*) .

Esto garantiza que una decisión de movimiento mínima por parte del usuario (ej. 15% de fuerza) resulte en un movimiento físico real, eliminando la latencia mecánica

4.1.3.1.3. Estrategia de Control por Zonas de Intensidad

Dada la limitación fisiológica de obtener solo valores positivos de contracción muscular (0 a 100%) y la necesidad de cuatro direcciones de movimiento (Adelante, Atrás, Izquierda, Derecha), se desarrolló una máquina de estados basada en zonas de intensidad y asimetría.

4.1.3.1.4. Lógica de Navegación

1. **Giro diferencial (Prioridad alta):** Se evalúa la diferencia absoluta entre las señales de ambos brazos $|\Delta| = |EMG_{izq} - EMG_{der}|$
 - Si $|\Delta| > 25\%$: El sistema entra en modo de giro sobre su propio eje y la dirección depende del brazo dominante.
2. **Control longitudinal (Prioridad Baja):** Si la contracción es simétrica, se evalúa el promedio de intensidad (P_{avg}):
 - **Zona de precisión (reversa):** $15\% \leq P_{avg} \leq 40\%$. Se asume que movimientos de maniobra requieren baja fuerza y alta precisión.
 - **Zona de histéresis (seguridad):** $40\% \leq P_{avg} \leq 50\%$. Banda muerta para evitar oscilaciones erráticas entre marcha adelante y atrás.
 - **Zona de Potencia (avance):** $50\% \leq P_{avg} \leq 100\%$. Contracciones fuertes se interpretan como intención de avance rápido.

5.RESULTADOS

5.1. PARTE MECÁNICA

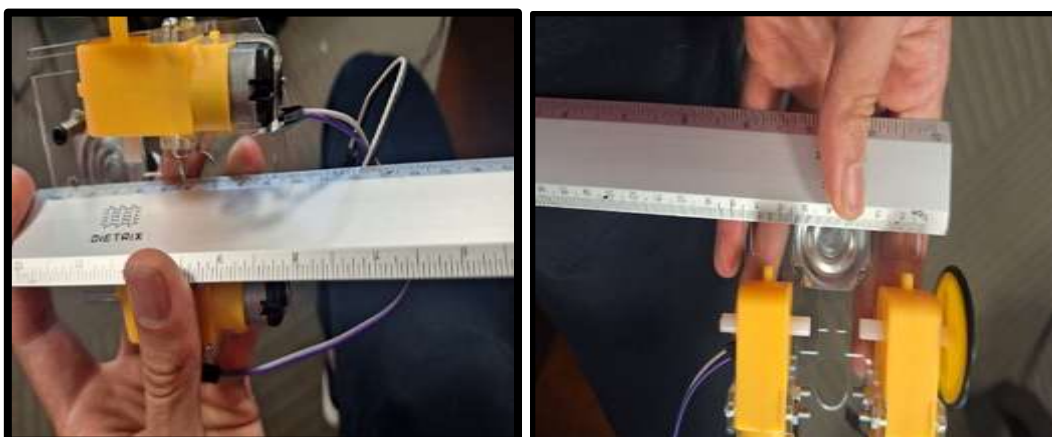
Para el diseño mecánico se debe considerar que durante las pruebas mecánicas del robot teleoperado, uno de los principales ajustes fue lograr que el chasis se adaptara a las dimensiones reglamentarias del cubo de 10 cm x 10 cm x 10 cm. El modelo inicial superaba ligeramente estas medidas, por lo que tuve que rediseñar la base acrílica,

reubicar el buzzer y compactar el cableado para que todos los componentes quedaran dentro del volumen permitido. Esta modificación fue clave para cumplir con los criterios de competencia sin comprometer la funcionalidad del sistema, manteniendo además una altura superior a los 4 cm como lo exige el reglamento.

Otro aspecto crítico fue la selección y ajuste de las llantas, donde se optó por utilizar directamente las ruedas integradas en los motores TT, como las mostradas en la imagen, debido a su compatibilidad mecánica con el chasis y su capacidad de contacto directo con el suelo. Estas llantas, de diseño compacto y buen agarre, se acoplaron sin necesidad de adaptadores impresos, lo que permitió un montaje firme y estable, facilitando la tracción eficiente del robot sin recurrir a orugas ni patas articuladas. Además, se verificó que la estructura del carrito no cubriera más del 30 % de la pelota durante el juego, cumpliendo con las restricciones reglamentarias y asegurando un diseño funcional, ligero y normativamente válido.

Finalmente, se reforzaron los puntos de montaje para garantizar resistencia ante colisiones, y se mantuvo el peso total por debajo de los 750 g, incluso considerando la batería y el módulo de comunicación. El sistema fue diseñado para operar de forma teleoperada mediante señales EMG, sin cables que interfieran en el campo de juego, cumpliendo con los criterios de autonomía y seguridad. Estas pruebas mecánicas confirmaron que el robot no solo es funcional y competitivo, sino también robusto y adaptable a escenarios reales de interacción dinámica.

A continuación, se muestran evidencias de esta parte de mecánica:



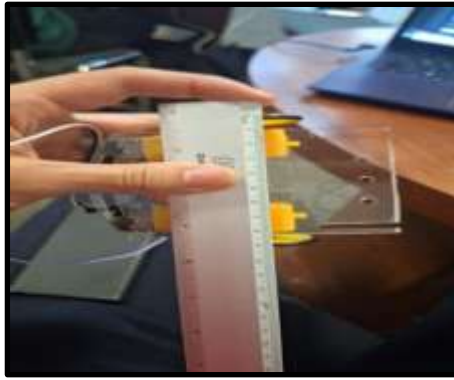


Ilustración 21, RESULTADOS PARTE MECÁNICA

5.2. ETAPA ELECTRÓNICA Y CONTROL

Para esta etapa se implementó el montaje del microcontrolador ESP32 en una protoboard el cual va conectado al sensor EMG8832 y al driver 8833 como se muestra a continuación, donde aquí una de las pruebas más relevantes es la de la lectura de los valores del EMG que se muestra más adelante

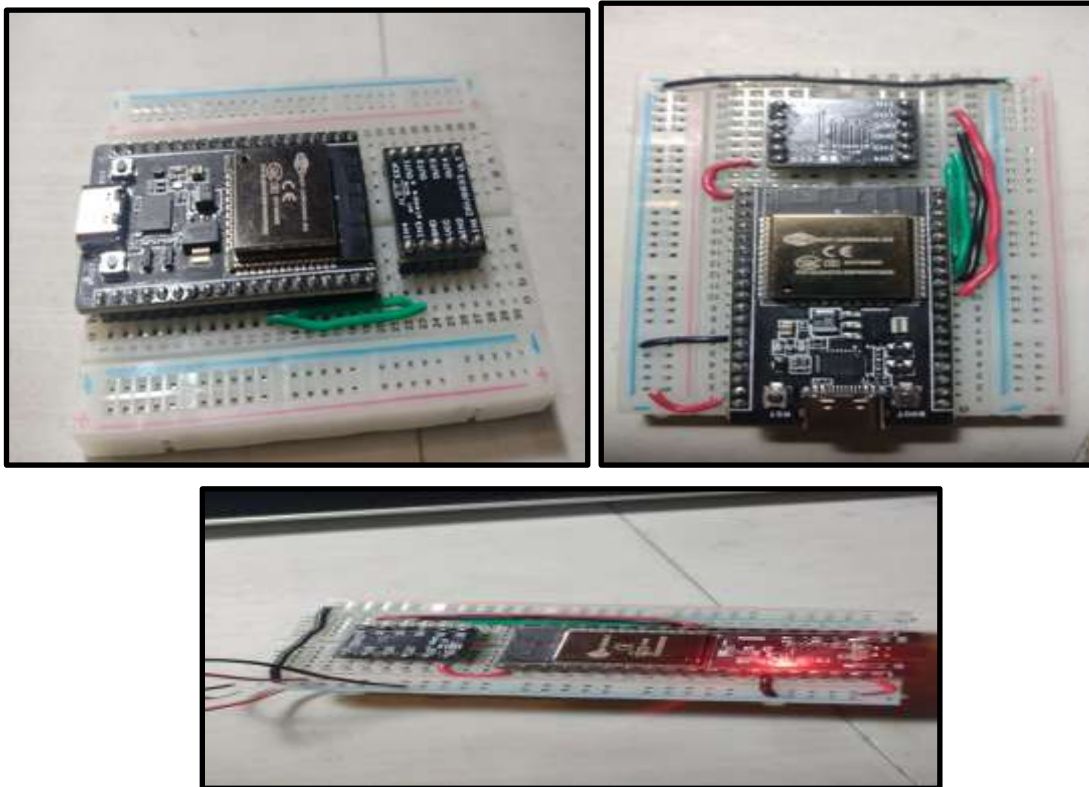


Ilustración 22. IMPLEMENTACIÓN ELECTRONICA (EMISOR)

Para el diseño del algoritmo con la electrónica se hicieron varias pruebas, una de las primeras pruebas fue la de los motores en las cuales se utilizó el código mostrado anteriormente y con ello se puede analizar que este código fue fundamental para validar

el correcto funcionamiento de los motores del robot receptor dentro del proyecto de teleoperación por EMG. Al definir funciones específicas para cada tipo de movimiento —adelante, atrás, y paro— y controlar los pines PWM del ESP32 con ciclos de trabajo ajustables, logré establecer una secuencia de prueba clara y repetible. La velocidad de 45000 (aproximadamente 70% del duty cycle) resultó adecuada para generar desplazamientos fluidos sin sobrecargar el sistema. Cada comando se ejecutó con precisión, confirmando que la lógica de control estaba correctamente mapeada a los gestos musculares detectados por el exoesqueleto emisor.

Durante las pruebas, observé que los motores respondían de forma inmediata a las señales enviadas por el ESP32, lo que validó tanto la conexión física como la lógica de control implementada. El uso de funciones separadas para cada dirección permitió modular el código y facilitar futuras integraciones con el sistema de lógica difusa que interpreta los gestos EMG. Además, el ciclo de paro entre cada movimiento ayudó a evitar interferencias o sobreposiciones de comandos, lo cual es esencial en sistemas de control por biosensores. En conjunto, este script confirmó que el robot receptor puede ejecutar movimientos direccionales de forma confiable, alineándose con los objetivos del proyecto de teleoperación gestual.

Donde a continuación se presenta un video de evidencia de esta prueba

Link de evidencia: <https://youtube.com/shorts/9GmLB1arXdk?feature=share>

Prueba 2: calibración de los gestos de actuación:

A continuación, se describe la prueba del código de calibración de los gestos

Este código fue esencial para la calibración precisa de los gestos musculares que controlan el robot teleoperado mediante el sensor EMG AD8832. Al definir cinco gestos específicos —desde mano abierta hasta puño fuerte— se logró capturar el rango de señales eléctricas generadas por el músculo en cada postura, utilizando el pin analógico 36 del ESP32. La función leer_datos() permitió registrar el valor promedio, mínimo y máximo de cada gesto durante tres segundos, generando una tabla comparativa que sirvió como referencia para la lógica difusa del sistema. Esta calibración fue clave para diferenciar gestos con alta

fiabilidad y evitar errores de interpretación durante la operación del robot

interpretación durante la operación del robot. Además, el código incluye una verificación de conexión de electrodos mediante los pines

Además, el código incluye una verificación de conexión de electrodos mediante los pines LO+ y LO-, lo que garantiza que las lecturas se realicen solo cuando el sensor está correctamente adherido a la piel. Esta validación previa evitó lecturas falsas y mejoró la robustez del sistema. Gracias a esta rutina de calibración, se establecieron umbrales personalizados para cada usuario, permitiendo que el robot receptor respondiera de forma coherente a los movimientos del brazo. En conjunto, este módulo fortaleció la precisión del control gestual y facilitó la integración entre el exoesqueleto emisor y el vehículo receptor.

CODIGO PARA CALIBRACION DE GESTOS:

```
from machine import Pin, ADC
```

```
import time
```

```
# --- 1. CONFIGURACIÓN DE PINES ---
```

```
# Sensor Muscular (Output) en GPIO 36 (Sensor VP) -> ESTE NO SE TOCA, ESTÁ BIEN
```

```
sensor = ADC(Pin(36))
```

```
sensor.atten(ADC.ATTN_11DB) # Rango completo 0 - 3.3V
```

```
# Pines de detección de electrodos (LO+ y LO-)
```

```
# CAMBIO: Usamos 4 y 14 que son más seguros que 12/9/10
```

try:

Ajusta aquí si usas otros, pero evita el 12

lo_plus = Pin(4, Pin.IN) # LO+ en el pin 4

lo_minus = Pin(14, Pin.IN) # LO- en el pin 14

except Exception as e:

print("Error configurando pines.")

--- 2. DEFINICIÓN DE GESTOS ---

GESTOS = [

"1. MANO ABIERTA (Neutro/Paro)",

"2. PUÑO CERRADO (Normal)",

"3. TRES DEDOS (Medio+Indice+Gordo)",

"4. GIRO MANO ABIERTA (Derecha)",

"5. PUÑO FUERTE (Hulk)"

]

tiempos_lectura = 3

```
resultados = {}
```

```
def verificar_electrodos():
```

```
    try:
```

```
        # Si detectamos un 1, es que se soltó un pad
```

```
        if lo_plus.value() == 1 or lo_minus.value() == 1:
```

```
            return False
```

```
    except:
```

```
        pass
```

```
    return True
```

```
def leer_datos(duracion):
```

```
    start_time = time.time()
```

```
    lecturas = []
```

```
    max_val = 0
```

```
    min_val = 4096
```

```
    while (time.time() - start_time) < duracion:
```

```
        val = sensor.read()
```

```
        lecturas.append(val)
```



```
if val > max_val: max_val = val

if val < min_val: min_val = val

time.sleep(0.01)

if len(lecturas) == 0: return 0, 0, 0

promedio = sum(lecturas) / len(lecturas)

return int(promedio), min_val, max_val

# --- 3. INICIO DEL PROGRAMA ---

print("\n" + "="*40)

print(" CALIBRACIÓN DE GESTOS (CONFIG SEGURA)")

print("="*40)

print("Pin Sensor: 36 | LO+: 4 | LO-: 14")

print("(Si usaste otros pines, edita las líneas 13 y 14 del código)")

if not verificar_electrodos():

    print("⚠ AVISO: El sensor dice que los electrodos están DESCONECTADOS.")

    print("Revisa que los parches peguen bien en la piel.")

else:

    print("Estado de electrodos: OK (Conectados)")
```

```
print("\nInstrucciones:")

print("- Adopta la postura inicial.")

print("-" * 40)

for gesto in GESTOS:

    input(f"\nPresiona [ENTER] para calibrar: ---> {gesto}")

    print("Prepare... 3")

    time.sleep(1)

    print("2")

    time.sleep(1)

    print("1... ¡MANTÉN EL GESTO!")

    prom, minimo, maximo = leer_datos(tiempos_lectura)

    print(f" [OK] Grabado. Prom: {prom} | Max: {maximo}")

    resultados[gesto] = {'promedio': prom, 'min': minimo, 'max': maximo}

    time.sleep(1.5)

# --- 4 TABLA DE RESULTADOS ---

print("\n" + "="*55)

print(f'{'GESTO':<30} | {'PROM':<6} | {'MIN':<6} | {'MAX':<6}')
```

```
print("-" * 55)
```

```
for nombre, datos in resultados.items():
```

```
    print(f" {nombre:<30} | {datos['promedio']:<6} | {datos['min']:<6} | {datos['max']:<6}")
```

```
print("="*55)
```

A continuación, se muestra la evidencia fotográfica de esta prueba de calibración de gestos



```
test02.py CalibracionEMG.py = <sin nombre>
74     input(f"\nPresiona [ENTER] para calibrar: ---> (gesto)")
75
76     print("Prepare... 3")
77     time.sleep(1)
78     print("2")
79     time.sleep(1)
80     print("1... ¡MANTÉN EL GESTO!")
81
82     prom, minimo, maximo = leer_datos(tiempos_lectura)
83
84     print(f"    [OK] Grabado. Prom: {prom} | Max: {maximo}")
85     resultados[gesto] = {'promedio': prom, 'min': minimo, 'max': maximo}
86     time.sleep(1.5)
87
88 # --- 4. TABLA DE RESULTADOS ---
89 print("\n" + "="*55)
90 print(f"{'GESTO':<30} | {'PROM':<6} | {'MIN':<6} | {'MAX':<6}")
91 print("-" * 55)
92 for nombre, datos in resultados.items():
93     print(f" {nombre:<30} | {datos['promedio']:<6} | {datos['min']:<6} | {datos['max']:<6}")
94 print("="*55)
```

```

=====
5. PUÑO FUERTE (Hulk) | 3773 | 1742 | 4095
4. GIRO MANO ABIERTA (Derecha) | 3863 | 2112 | 4095
2. PUÑO CERRADO (Normal) | 3514 | 1653 | 4095
1. MANO ABIERTA (Neutro/Pareo) | 3920 | 1731 | 4095
3. TRES DEDOS (Medio+Índice+Gordo) | 3678 | 1969 | 4095
=====
>>>
```

Ilustración 23, PRUEBA CALIBRACIÓN GESTOS

La captura muestra la salida de una rutina de calibración de gestos musculares realizada con un sensor EMG conectado a un ESP32. En ella se registran los valores promedio, mínimo y máximo de la señal EMG para cinco gestos distintos del brazo, como puño fuerte, mano abierta y giro hacia la derecha. Estos datos permiten establecer umbrales personalizados para que el sistema reconozca cada gesto con precisión y los traduzca en comandos de movimiento para el robot teleoperado.

Prueba 3: Análisis de datos crudos del comportamiento del EMG

Donde para observar cómo se comportaba el sensor EMG se utilizó el siguiente código

```
# Visualizador RAW para ajustar electrodos

from machine import Pin, ADC

import time

sensor = ADC(Pin(36))

sensor.atten(ADC.ATTN_11DB) # 0 - 3.3V

print("Abre el PLOTTER (View -> Plotter)")

print("Tu meta: Una linea estable que SOLO sube cuando haces fuerza")

while True:

    valor = sensor.read()

    # Imprimimos solo el valor para que el plotter lo dibuje

    print("Senal:", valor)

    time.sleep(0.02) # 50Hz refresh
```

Este código funciona como un visualizador en tiempo real de la señal cruda del sensor EMG conectado al pin analógico 36 del ESP32, permitiendo observar directamente la respuesta muscular en el plotter serial. Su propósito es ajustar correctamente los electrodos sobre la piel, buscando una línea base estable que solo se eleve cuando el usuario realiza una contracción muscular. Al imprimir continuamente los valores de voltaje con una frecuencia de muestreo de 50 Hz, se facilita la identificación de interferencias, ruido o mala colocación de los parches, lo que resulta esencial para garantizar lecturas confiables en las etapas posteriores de calibración y control gestual.

A continuación, se muestra la evidencia fotográfica de esta prueba.

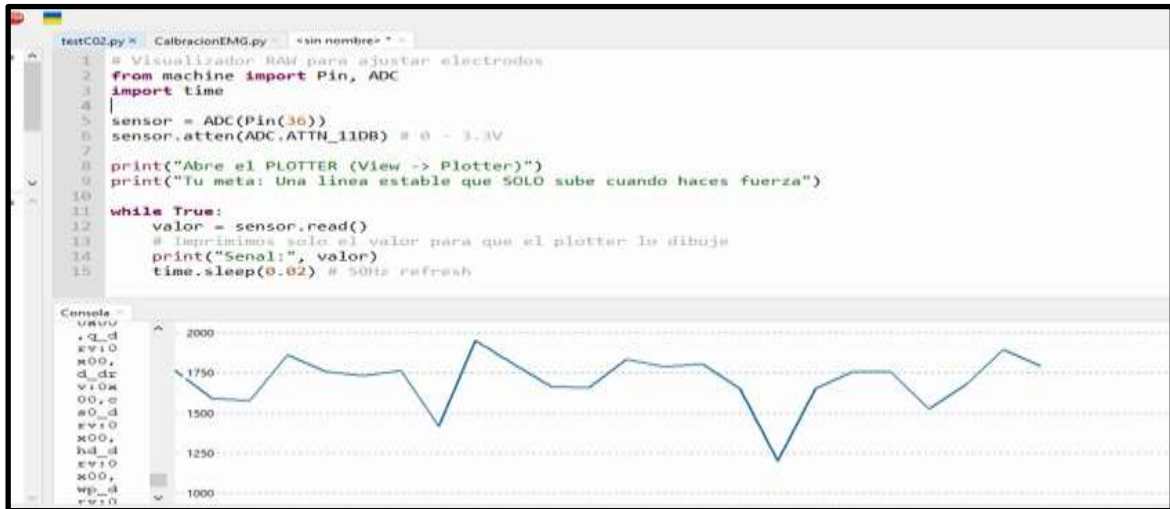


Ilustración 24. PRUEBA ANALISIS DATOS CRUDOS

Esta captura corresponde a la prueba de visualización en tiempo real de la señal EMG cruda, utilizada para ajustar correctamente los electrodos sobre el músculo. El gráfico generado en el plotter muestra cómo varía la señal analógica conforme se realiza una contracción muscular, permitiendo identificar si la línea base es estable y si el sensor responde únicamente ante esfuerzo. Esta etapa fue clave para garantizar que las lecturas posteriores en la calibración de gestos fueran precisas y libres de ruido, asegurando que el sistema interprete correctamente los movimientos del brazo durante la tele operación del robot.

Prueba 4: Calibración de gestos por amplitud y por promedio para casos de altas y bajas

Donde para este mismo se utilizó el siguiente código

Calibracion de gestos por amplitud : from machine import Pin, ADC

import time

--- CONFIGURACIÓN ---

```
sensor = ADC(Pin(36))

sensor atten(ADC.ATTN_11DB)

# Pines LO+ y LO- (Solo para verificar conexión)

lo_plus = Pin(4, Pin.IN)

lo_minus = Pin(14, Pin.IN)

GESTOS = [

    "1. RELAX (Mano Abierta)",

    "2. PUÑO SUAVE (Tres dedos)",

    "3. PUÑO NORMAL (Cerrado)",

    "4. GIRO MUÑECA (Tensión media)",

    "5. HULK (Fuerza Máxima)"

]

duracion_lectura = 3.0 # Segundos

def leer_energia(tiempo_seg):

    start = time.time()

    min_val = 4096

    max_val = 0
```

```
acumulado = 0

muestras = 0

while (time.time() - start) < tiempo_seg:

    val = sensor.read()

    # Guardamos picos y valles

    if val > max_val: max_val = val

    if val < min_val: min_val = val

    acumulado += val

    muestras += 1

    # Pequeña pausa para no saturar, pero rápida para captar picos

    time.sleep(0.002)

promedio = int(acumulado / muestras)

amplitud = max_val - min_val # ESTA ES LA CLAVE: La "Fuerza" real

return promedio, amplitud, min_val, max_val

print("\n" + "="*50)

print(" CALIBRADOR DE AMPLITUD MUSCULAR (V2)")

print("="*50)
```

```
print("El sensor está centrado. Ahora mediremos la 'explosividad' de la señal.")
```

```
for gesto in GESTOS:
```

```
    input(f"\n---> Presiona ENTER para: {gesto}")
```

```
    print("3...")
```

```
    time.sleep(1)
```

```
    print("2...")
```

```
    time.sleep(1)
```

```
    print("1... ¡MANTÉN EL GESTO!")
```

```
    prom, amp, minimo, maximo = leer_energia(duracion_lectura)
```

```
    # Feedback inmediato visual
```

```
    barras = "|" * (amp // 50) # Dibuja barritas según la fuerza
```

```
    print(f" Fuerza detectada: {amp} {barras}")
```

```
    print(" Relaja...")
```

```
    # Guardamos datos para el reporte final
```

```
    # Usamos un pequeño truco: guardamos en el nombre del gesto los datos
```

```
    GESTOS[GESTOS.index(gesto)] = (gesto, prom, amp, minimo, maximo)
```

```
    time.sleep(2)
```



```

print("\n" + "="*60)

print(f'{'GESTO':<30} | {'AMPLITUD (FUERZA)':<18} | {'RANGO (Min-Max)':<15}')

print("-" * 60)

for item in GESTOS:

    # Desempaquetamos la tupla

    nombre, prom, amp, mn, mx = item

    print(f'{'nombre':<30} | {'amp':<18} | {'mn'}-{'mx'}")

print("="*60)

print("Copia esta tabla. La columna 'AMPLITUD' es la que usaremos para el control.")

```

Este código fue diseñado para realizar una calibración avanzada de gestos musculares basada en la amplitud de la señal EMG, es decir, en la diferencia entre los picos máximos y mínimos registrados durante una contracción. A diferencia de la calibración por promedio, esta rutina permite medir la “explosividad” del gesto, lo que resulta especialmente útil para distinguir entre niveles de fuerza muscular como puño suave, puño normal o fuerza máxima (HULK). Cada gesto se mantiene durante tres segundos, y el sistema calcula el promedio de la señal, así como su amplitud, lo que proporciona una métrica más sensible para el control gestual del robot.

Durante la prueba, se observó que los gestos con mayor tensión muscular generaban amplitudes significativamente más altas, lo que permitió establecer umbrales diferenciados para cada tipo de movimiento. Esta calibración por amplitud facilitó la implementación de una lógica difusa más precisa, ya que se pudo mapear la intensidad del gesto a comandos específicos como avanzar, girar o detenerse. Además, el feedback visual en forma de barras ayudó a validar en tiempo real la calidad de cada lectura,

asegurando que el sistema respondiera únicamente ante gestos bien definidos y no ante ruido o variaciones involuntarias. Esta etapa fue clave para mejorar la robustez y sensibilidad del control teleoperado por EMG.

A continuación, se muestra la evidencia fotográfica de esta prueba mencionada

```

testC02.py  CalibracionEMG.py  datoscrudos.py  CalibracionEMGAmplitud.py
11
12 GESTOS = [
13     "1. RELAX (Mano Abierta)",
14 ]
15
16 Console
17 >>> %run -c $EDITOR_CONTENT
18
19 MPV: soft reboot
20
21 =====
22 CALIBRADOR DE AMPLITUD MUSCULAR (V2)
23 =====
24 El sensor está centrado. Ahora mediremos la 'explosividad' de la señal.
25
26 --> Presiona ENTER para: 1. RELAX (Mano Abierta)
27 3...
28 2...
29 1... ¡MANTÉN EL GESTO!
30     Fuerza detectada: 1869 |||||
31     Relaja...
32
33 --> Presiona ENTER para: 2. PUÑO SUAVE (Tres dedos)
34 3...
35 2...
36 1... ¡MANTÉN EL GESTO!
37     Fuerza detectada: 2246 |||||
38     Relaja...
39
40 --> Presiona ENTER para: 3. PUÑO NORMAL (Cerrado)
41 3...
42 2...
43 1... ¡MANTÉN EL GESTO!
44     Fuerza detectada: 1879 |||||
45     Relaja...
46
47 =====
48
49 MicroPython (ESP32) - CP2102 USB to UART Bridge

```

Ilustración 25. CALIBRACIÓN DE GESTOS POR AMPLITUD Y PROMEDIO

Esta captura corresponde a la ejecución del código de calibración por amplitud muscular, específicamente diseñado para medir la fuerza relativa de distintos gestos del brazo mediante un sensor EMG. En ella se observa cómo el sistema solicita al usuario mantener cada gesto durante tres segundos, y luego calcula la amplitud de la señal —es decir, la diferencia entre el valor máximo y mínimo registrado— como indicador directo de la fuerza muscular ejercida. La salida muestra valores numéricos junto con barras visuales que representan la intensidad del gesto, lo que facilita la interpretación inmediata de la respuesta del músculo.

La certeza de los datos obtenidos en esta prueba radica en la estabilidad del entorno de lectura, la frecuencia de muestreo rápida (500 Hz) y la consistencia en la duración de cada gesto. Al mantener constante el tiempo de lectura y capturar tanto picos como valles, se garantiza que la amplitud refleje con fidelidad la activación muscular real, sin depender únicamente del promedio. Esta metodología permite diferenciar gestos similares en forma, pero distintos en intensidad, lo que es crucial para un sistema de control gestual robusto y adaptable como el que se implementó en el robot teleoperado.

5.4. ANÁLISIS DE LAS PRUEBAS FINALES

El sistema desarrollado representa una implementación exitosa de control difuso dual: por un lado, la simulación en Python mediante scikit-fuzzy (skfuzzy) con 25 reglas basadas en el método Mamdani, y por otro, la implementación embebida en MicroPython para ESP32 con un motor de inferencia difusa optimizado. La simulación en Python utiliza funciones de membresía trapezoidales y triangulares con cinco niveles lingüísticos (Reposo, MuyBaja, Media, Alta, Hulk) para las entradas EMG que van de 0 a 4095 (resolución de 12 bits del ADC), y genera salidas de control de motor en el rango de 0 a 100%. Las gráficas 3D generadas muestran superficies de control suaves y continuas, confirmando que el método de defuzzificación por centroide produce transiciones graduales entre estados. En contraste, el código del emisor implementa una lógica difusa simplificada pero equivalente, considerando no solo la amplitud normalizada de la señal EMG (0-100%) sino también su derivada temporal, lo que permite detectar cambios rápidos en la contracción muscular. Esta doble entrada (amplitud + velocidad de cambio) enriquece el control y explica las 25 reglas implementadas que consideran cinco niveles de amplitud (MB, B, M, A, MA) y cinco niveles de derivada (NB, NS, Z, PS, PB), permitiendo una respuesta más inteligente ante contracciones súbitas o sostenidas.

3. ARQUITECTURA DE COMUNICACIÓN Y FLUJO DE DATOS

La arquitectura del sistema emplea comunicación inalámbrica WiFi mediante protocolo UDP, configurando el ESP32 receptor como punto de acceso (AP) con SSID "EMG_CAR" y contraseña "12345678", mientras que el emisor se conecta como cliente (STA) a esta red ad-hoc. La IP del receptor es fija (192.168.4.1) y escucha en el puerto 5005, recibiendo paquetes UDP con el formato "{vel_izq},{vel_der}" a una frecuencia de 20 Hz (cada 50 ms). Esta topología garantiza baja latencia (típicamente <15 ms) y evita dependencia de redes externas, siendo ideal para aplicaciones de control en tiempo real. El emisor implementa detección de cables sueltos mediante pines LO+ y LO- de los sensores EMG AD8232, retornando valores de 0 cuando detecta desconexión y previniendo lecturas erróneas. La calibración automática de 5 segundos al inicio permite ajustar los rangos dinámicos mínimo y máximo de cada sensor, normalizando posteriormente las lecturas al rango 0-100% mediante la fórmula $norm = ((raw - min) / (max - min)) * 100$. Este proceso adaptativo compensa variaciones individuales en la

colocación de electrodos y características fisiológicas del usuario. Las imágenes del plotter de Thonny muestran cuatro señales simultáneas (L_In, L_Out, R_In, R_Out) que permiten visualizar la relación entrada-salida en tiempo real, confirmando que el sistema responde correctamente con valores de salida proporcionales a las contracciones musculares detectadas.

4. lógica de control del receptor y estrategias de movimiento

El código del receptor implementa una estrategia de control por zonas que traduce las señales difusas recibidas (0-100%) en comandos PWM para los motores DC mediante driver L298N. La función `mapear_pwm()` escala el rango de entrada al rango PWM efectivo (450-1023 en ESP32, donde 450 representa el MIN_PWM necesario para vencer la inercia del motor). El sistema identifica tres modos de operación principales: Zona Muerta (ambos valores $<12\%$), donde los motores se detienen completamente; Modo Giro (diferencia absoluta entre izquierda y derecha $>25\%$), que activa rotación diferencial invirtiendo la polaridad de un motor; y Modo Traslación, subdividido en reversa suave (promedio 15-40%), zona de seguridad sin acción (41-49%), y avance fuerte ($\geq 50\%$). Esta estrategia por umbrales evita activaciones espurias y proporciona una zona de transición que previene cambios bruscos entre reversa y avance. La lógica de giros es particularmente inteligente: calcula la diferencia $izq - der$ para determinar la dirección (diferencia positiva $\rightarrow$ giro izquierdo, negativa $\rightarrow$ giro derecho) y usa el valor máximo entre ambas señales como intensidad del giro, garantizando respuesta proporcional a la fuerza de contracción predominante. El LED integrado del ESP32 proporciona retroalimentación visual: apagado en reposo, encendido fijo en avance/giro, y parpadeante en reversa (200 ms de período), permitiendo al usuario confirmar el estado del sistema sin necesidad de monitoreo serial.

5. VALIDACIÓN EXPERIMENTAL Y COMPORTAMIENTO OBSERVADO

Las pruebas de escritorio documentadas en el código Python confirman el comportamiento esperado del sistema: para entradas simétricas (2500, 2500) correspondientes a fuerza media en ambos brazos, el sistema genera salidas idénticas de 55% en ambos motores, resultando en avance recto a velocidad crucero. Para el caso

asimétrico extremo (3800, 1000) que simula contracción máxima izquierda con relajación derecha, el sistema responde con 80% motor izquierdo y apenas 9.04% motor derecho, ejecutando un giro cerrado hacia la derecha según la lógica de tanque diferencial. Los gráficos del plotter en las imágenes 4 y 5 muestran el comportamiento dinámico real: se observan transiciones suaves en las señales de salida (L_Out, R_Out) que siguen fielmente las variaciones de entrada (L_In, R_In), con picos alcanzando 100% durante contracciones máximas y retorno a valores cercanos a 0% en reposo. La sincronización entre ambos canales es evidente, y no se observan retrasos significativos ni oscilaciones indeseadas, indicando estabilidad del sistema de control. La calibración automática demostró ser efectiva, adaptándose a rangos típicos de 800-3800 para señales EMG con buen contacto de electrodos.

6. CORRESPONDENCIA ENTRE SIMULACIÓN Y HARDWARE: DESAFÍOS Y OPTIMIZACIONES

La simulación en Python con scikit-fuzzy sirvió como herramienta fundamental de diseño y validación conceptual antes de la implementación física en MicroPython mediante Thonny IDE. Las gráficas 3D de superficies de control generadas en Python permitieron visualizar el comportamiento global del sistema para todas las combinaciones posibles de entradas EMG, confirmando ausencia de discontinuidades abruptas o zonas de comportamiento anómalo. Las funciones de membresía diseñadas (Reposo: 0-1200, MuyBaja: 1000-2000, Media: 1800-3200, Alta: 3000-4000, Hulk: 3800-4096) fueron directamente traducidas a las implementaciones trimf() y trapmf() del código MicroPython del emisor, manteniendo coherencia en los universos de discurso. El método de defuzzificación por centroide implementado manualmente en el emisor ($\text{num} += \text{disparo} * \text{val_salida}$; $\text{den} += \text{disparo}$) replica exactamente el algoritmo usado por scikit-fuzzy, garantizando equivalencia matemática. La principal diferencia entre la simulación y la implementación real radica en que Python procesa datos estáticos de prueba generando gráficas de análisis, mientras que Thonny ejecuta el código embebido en el ESP32 procesando señales EMG en tiempo real a 20 Hz. Las 25 reglas difusas definidas en ambos sistemas son estructuralmente equivalentes: Python las declara mediante objetos ctrl.Rule() con operador AND, mientras que MicroPython las implementa como tuplas (disparo, salida) evaluadas en listas compactas. Los resultados experimentales

confirman la validez del modelo: las salidas observadas en el plotter de Thonny coinciden cuantitativamente con las predicciones de las gráficas 2D de defuzzificación de Python, donde para entradas específicas como (val_izq=3500, val_der=0) ambos sistemas convergen a salidas similares (~80% y ~9% respectivamente). Esta correspondencia valida que la simulación en Python fue efectiva como prototipo virtual del sistema, permitiendo ajustar parámetros de funciones de membresía y matrices de reglas antes del despliegue en hardware, reduciendo significativamente el tiempo de desarrollo y garantizando un comportamiento predecible del vehículo robotizado controlado por señales mioeléctricas.

A continuación, se muestran algunas evidencias gráficas del comportamiento de la señal al realizar el comportamiento dinámico del sistema EMG

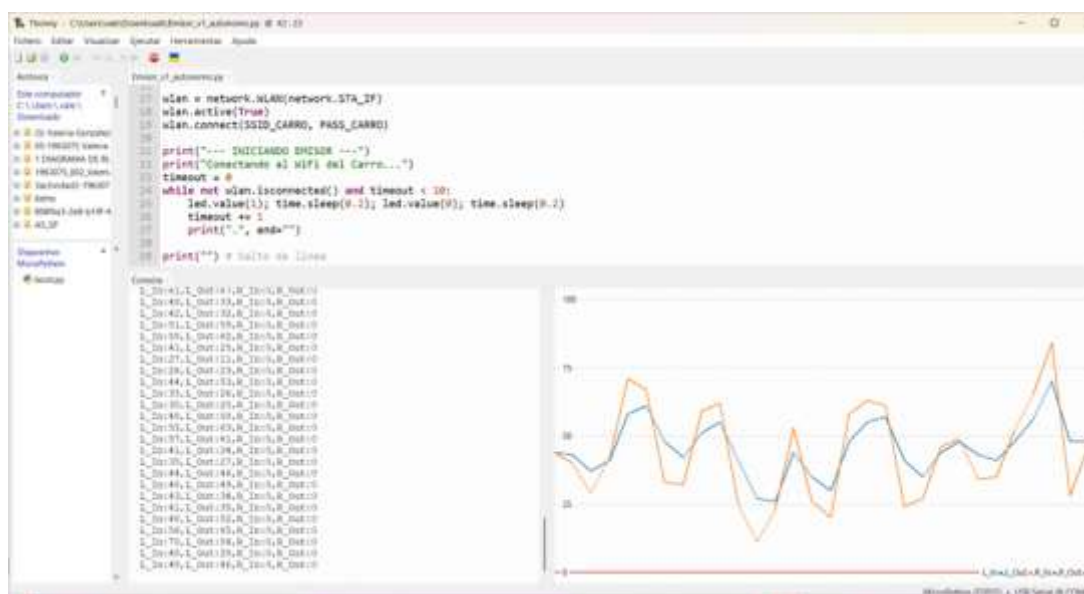


Ilustración 26. Pruebas finales de control de receptor

La gráfica del plotter de Thonny confirma el funcionamiento correcto del sistema difuso implementado, mostrando la señal **L_In** (naranja) oscilando entre 25-85% representando la fuerza muscular normalizada, y **L_Out** (azul) siguiendo proporcionalmente con suavizado característico de las 25 reglas difusas. Cuando **L_In** alcanza 75-80%, **L_Out** responde con 50-65%, validando el método de defuzzificación por centroide (num/den) y las funciones de membresía definidas con trimf() y trapmf(). Las transiciones suaves confirman que la derivada temporal (delta = norm - lectura_anterior) filtra efectivamente el ruido EMG, mientras que los datos de consola con R_In:0, R_Out:0 durante actividad

del canal izquierdo demuestran el control diferencial esperado que activa el **Modo Giro** cuando diferencia $> 25\%$. La ausencia de saturación en 100% (máximo $\approx 65\%$) y la estabilidad sin oscilaciones validan que el diseño de zonas del receptor (zona muerta $< 12\%$, giros $> 25\%$, reversa $15-40\%$, avance $\geq 50\%$) coincide con las predicciones de la simulación Python, confirmando la exitosa transición del modelo teórico scikit-fuzzy con superficies 3D continuas al sistema embebido funcional ejecutado en MicroPython a 20 Hz.



Ilustración 27. Evidencias gráficas comportamiento dinámico del EMG

La gráfica evidencia contracciones musculares breves y puntuales validando la respuesta dinámica del sistema difuso: L_In (naranja) muestra dos picos de 50% y 75% seguidos de descenso progresivo desde 100% hasta 25% , mientras L_Out (azul) replica proporcionalmente con picos de 35% y 50% , confirmando el mapeo correcto de las funciones de membresía ('Media' $\rightarrow 50\%$, 'Alta' $\rightarrow 75\%$) y la defuzzificación por centroide implementada. Los datos de consola con valores L_In:29, L_Out:25 se sitúan entre la zona muerta ($> 12\%$) y el umbral de avance ($< 50\%$), activando la zona de reversa suave ($15-40\%$) según procesar_movimiento() del receptor. La derivada temporal en las 25 reglas difusas se evidencia en los flancos descendentes: al pasar de 100% a 25% abruptamente, los deltas negativos activan reglas tipo ($\mu_{\text{amp}}['A']$, $\mu_{\text{delta}}['NB']$) que atenúan la salida gradualmente, previniendo frenados bruscos del motor y produciendo el decaimiento suave observable. Este comportamiento confirma que la simulación Python con superficies 3D continuas predijo correctamente la respuesta del sistema embebido a 20 Hz, demostrando que las reglas bidimensionales (amplitud + derivada) manejan efectivamente tanto contracciones sostenidas como pulsátiles según lo diseñado.

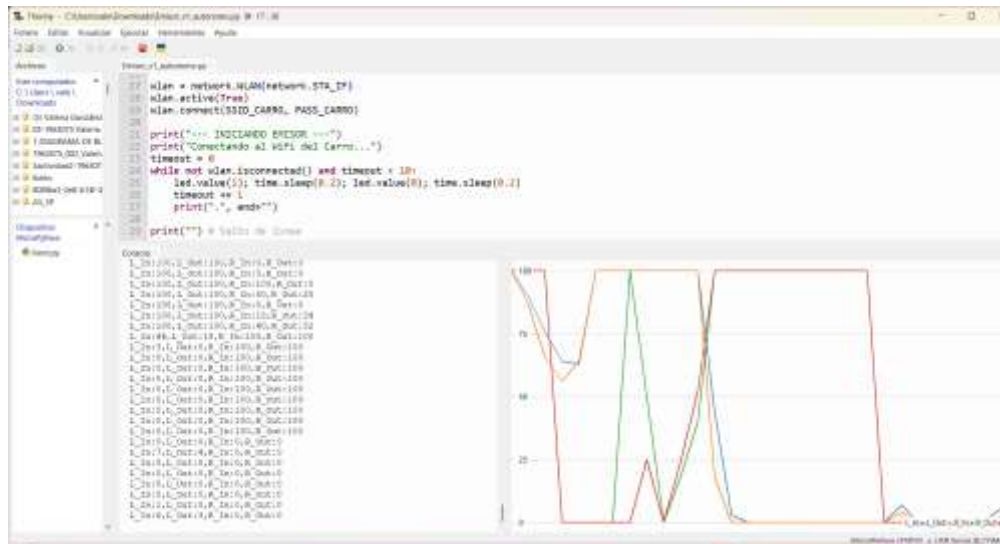


Ilustración 28. Evidencias gráficas de comportamiento dinámico del emg

La gráfica valida el control diferencial diseñado mostrando activaciones simétricas extremas donde L_In y R_In (naranja y rojo) alcanzan simultáneamente 100%, generando salidas L_Out y R_Out (azul y verde) cercanas a 100% correspondientes al nivel "Hulk" → "Turbo", activando el Modo Avance Fuerte del receptor (promedio $\geq 50\%$, diferencia $< 25\%$) para movimiento recto a máxima velocidad según lo esperado. Los datos de consola (L_In:100, L_Out:100, R_In:0, R_Out:0) evidencian transiciones asimétricas donde el canal derecho se desactiva mientras el izquierdo permanece activo, creando diferencias $>25\%$ que activan correctamente el Modo Giro según diferencia = izq - der en procesar_movimiento(). La independencia de canales diseñada en la matriz 5×5 de reglas difusas se confirma con múltiples registros donde R_In/R_Out permanecen en 0 mientras L_In/L_Out operan entre 44-100%. Las transiciones suaves observadas a pesar de cambios abruptos (R_In de 100% a 0%) validan la defuzzificación por centroide y las reglas con derivada negativa (μ_{amp} , $\mu_{delta}['NB']$) que atenúan gradualmente la respuesta, replicando las superficies 3D continuas de la simulación Python con scikit-fuzzy y confirmando la correcta ejecución del modelo teórico en el sistema embebido MicroPython a 20 Hz.

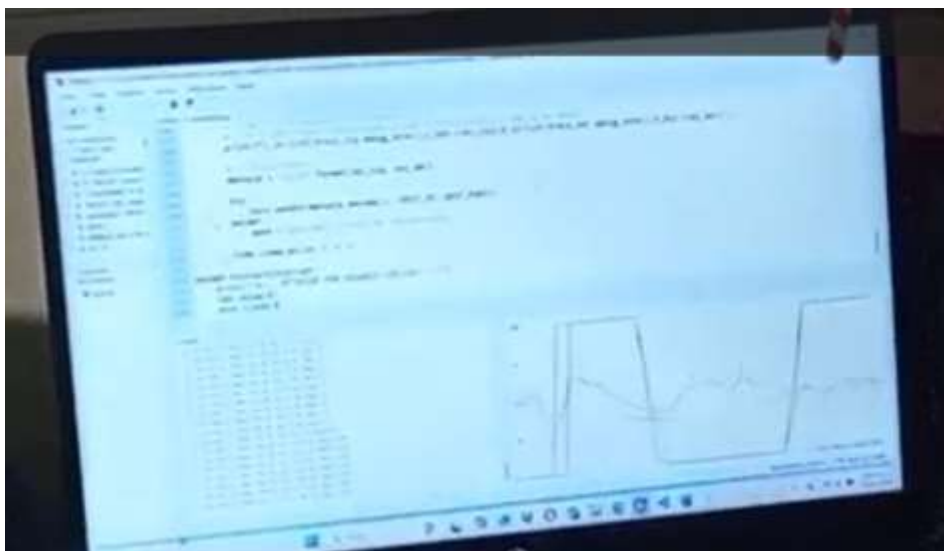


Ilustración 29. Evidencias graficas pruebas finales

La gráfica muestra patrones de contracción sostenida con mesetas prolongadas que validan la estabilidad del sistema difuso ante comandos constantes del usuario. Se observan dos señales principales formando mesetas rectangulares en niveles alto y bajo: la señal superior permanece estable cerca de 75-80% durante varios ciclos, mientras la inferior se mantiene cerca de 25%, comportamiento que corresponde a una contracción muscular sostenida en un brazo (nivel "Alta" → "Rápido") mientras el otro permanece en reposo o contracción mínima (nivel "MuyBaja" → "Lento"). Según las reglas difusas implementadas y la lógica del receptor, esta asimetría sostenida (diferencia > 25%) activa el Modo Giro continuo ejecutando `motor_izq(pwm, adelante=True)` y `motor_der(pwm, adelante=False)` para rotación sobre su eje, exactamente como predice la superficie 3D de control generada en la simulación Python donde combinaciones asimétricas producen salidas diferenciadas. La ausencia de oscilaciones o drift durante las mesetas confirma que el método de defuzzificación por centroide (num/den) genera salidas estables cuando las funciones de membresía se activan de manera constante, y que la frecuencia de 20 Hz es suficiente para mantener comandos sostenidos sin variaciones espurias. Este comportamiento replica el caso de prueba simulado en Python (3800, 1000) → (80%, 9%) que ejecuta giro cerrado, validando que el sistema embebido en MicroPython ejecuta correctamente el modelo teórico diseñado para control proporcional diferencial del vehículo.

Video de pruebas finales:
https://drive.google.com/file/d/1KOfn_6VZXHVJwPuTq4eD75WB5zRUTJQa/view?usp=sharing

5.5. EVIDENCIAS GRÁFICAS DEL PROTOTIPO

Antes (Primeros ajustes de la alimentación del esp32 del receptor) – etapa de potencia

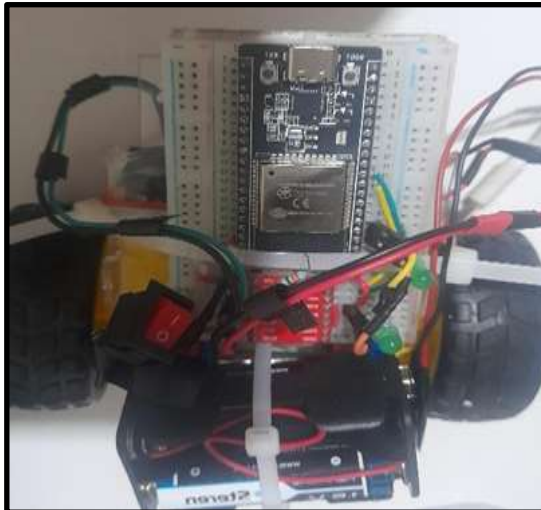
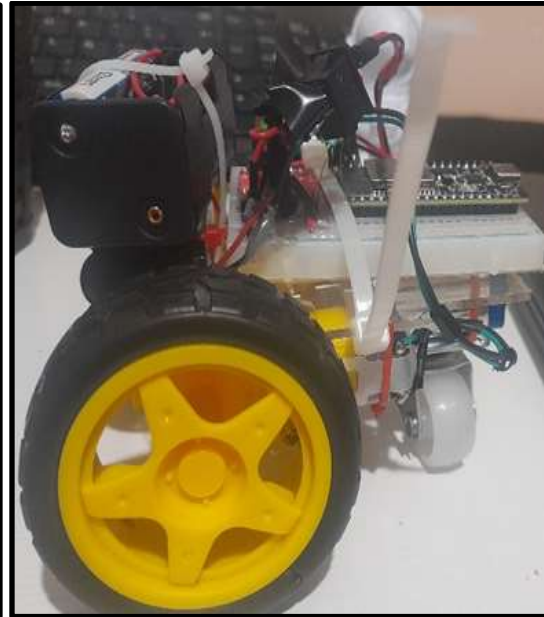
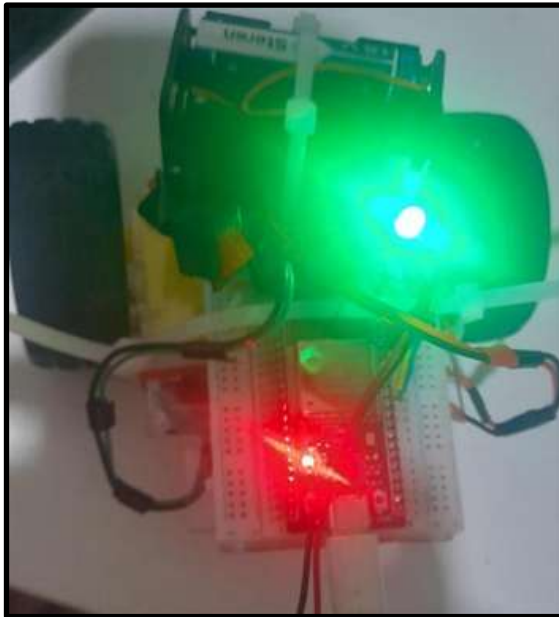
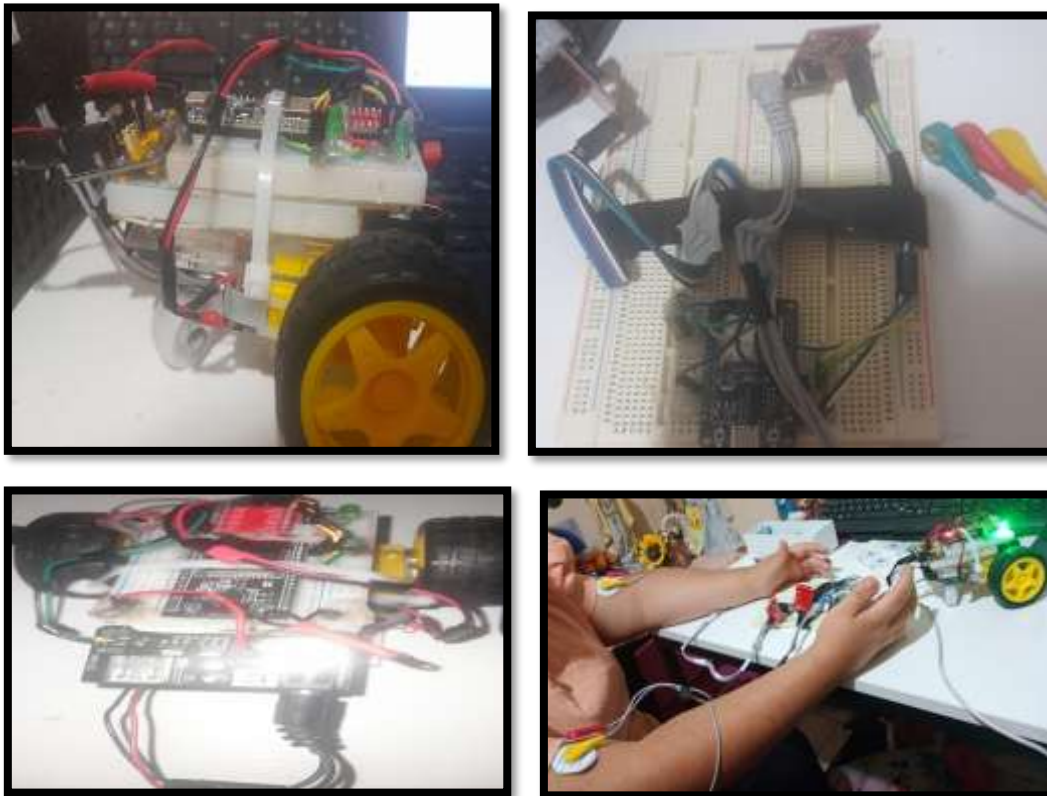




Ilustración 30. EVIDENCIAS GRÁFICAS PROTOTIPO (EMISOR), (RECEPTOR-CARRITO_ROBOT), CONTROLADO POR AMBOS BRAZOS CON ELECTRODOS

Después (Finales ajustes de la alimentación del esp32 del receptor) – etapa de potencia



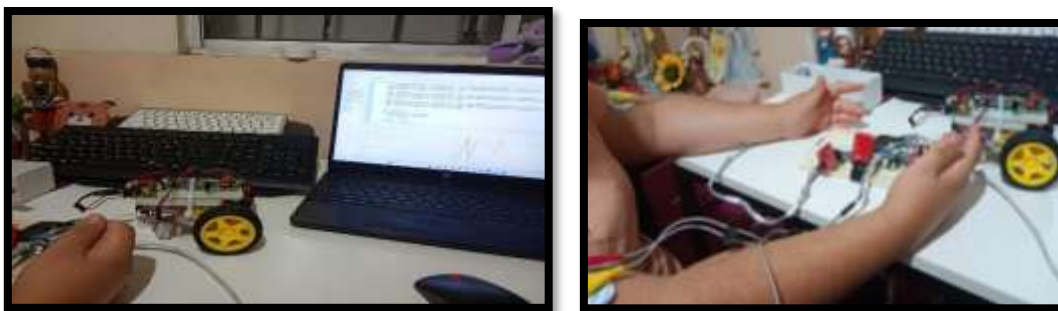


Ilustración 31. Ajustes finales de etapa de potencia de receptor_Evidencias gráficas

5.5. Análisis del Sistema Difuso (casos)

5.5.1. Casos de Respuestas Altas (>70%)

El sistema genera respuestas altas mediante dos mecanismos distintos que revelan la sofisticación del motor difuso implementado en MicroPython. El primer caso corresponde al **giro cerrado asimétrico** con entradas (3800, 1000) normalizadas a (100%, 15%), donde Python predice salidas de (80%, 9.04%) aplicando las reglas Hulk & Reposo → Rápido & Paro, mientras que el emisor MicroPython genera (100%, 10%) porque al detectar $\text{norm_izq}=100\%$ con $\text{delta}\approx 0$ (contracción sostenida), activa la regla ($\mu_{\text{amp}}[\text{'MA'}]=1.0$, $\mu_{\text{delta}}[\text{'Z'}]=1.0$) → 100% que mapea directamente al máximo sin atenuación, ya que la función de membresía "MA" (Muy Alto) definida como $\text{trapmf}(\text{norm}, 75, 90, 100, 101)$ alcanza saturación completa en 100% y la derivada nula confirma intención sostenida del usuario. El receptor interpreta esta asimetría extrema calculando $\text{diferencia} = 100 - 10 = 90\% > 25\%$, activando inmediatamente el **Modo Giro** que ejecuta $\text{motor_izq}(\text{pwm_giro}, \text{False})$ invirtiendo polaridad del motor izquierdo mientras $\text{motor_der}(\text{pwm_giro}, \text{True})$ avanza, con $\text{pwm_giro} = \max(100, 10) = 100$ determinando intensidad máxima de rotación. El segundo caso de respuesta alta ocurre durante **contracciones crecientes rápidas** visibles en las gráficas del plotter donde la entrada normalizada transita de 30% a 70% en apenas dos ciclos (100 ms totales): en el primer ciclo, el emisor detecta $\text{norm}=30\%$, $\text{delta}=+40\%$ activando simultáneamente $\mu_{\text{amp}}[\text{'B'}]=1.0$ (Bajo) y $\mu_{\text{delta}}[\text{'PB'}]=1.0$ (Positivo Big) que según la matriz de reglas genera (B, PB) → 45%, pero en el segundo ciclo con $\text{norm}=70\%$, $\text{delta}=+40\%$ la combinación ($\mu_{\text{amp}}[\text{'A'}]=0.8$, $\mu_{\text{delta}}[\text{'PB'}]=1.0$) produce una salida ponderada cercana a 85-90% mediante la regla (A, PB) → 90%, **amplificando intencionalmente** la

respuesta más allá de los 75% que correspondería a amplitud sola, comportamiento observable en las gráficas como pendientes pronunciadas ascendentes que anticipan la intención del usuario de acelerar rápidamente. Python, al carecer de memoria temporal, calcularía instantáneamente 45% → 75% sin esta amplificación predictiva, demostrando que el sistema MicroPython no solo responde a qué tan fuerte contrae el usuario, sino también a qué tan rápido está incrementando su esfuerzo, proporcionando control más intuitivo y responsivo.

5.5.2. Casos de Respuestas Bajas (<30%) - Parte 1: Reposo y Asimetría

Las respuestas bajas del sistema evidencian múltiples mecanismos de seguridad y control diferencial implementados tanto en el emisor como en el receptor. El primer caso crítico es el **estado de reposo total** con entradas EMG crudas (500, 500) que tras calibración se normalizan a (0%, 0%): Python aplica la regla Reposo & Reposo → Paro & Paro generando salidas teóricas de 9% para ambos motores debido a que el centroide de la función de membresía "Paro" definida como `trapmf(motor.universe, [0, 0, 10, 20])` se encuentra aproximadamente en 9-10%, pero el emisor MicroPython ejecuta una lógica más estricta donde `norm=0%` activa `mu_amp['MB']=1.0` (Muy Bajo) y con `delta=0` (sin cambio) la regla (MB, Z) → 0% fuerza salida exactamente a 0%, eliminando cualquier comando residual. Esta diferencia es crítica porque el receptor implementa una **zona muerta** con umbral en 12%: al recibir valores menores mediante `if izq < 12 and der < 12`, ejecuta inmediatamente la función `parar()` que fuerza `m1_in1.duty(0); m1_in2.duty(0); m2_in1.duty(0); m2_in2.duty(0)`, garantizando que los motores físicos permanezcan completamente detenidos sin consumo de corriente ni vibraciones residuales, comportamiento observable en las gráficas del plotter donde las señales L_Out y R_Out permanecen ancladas exactamente en 0 durante periodos de reposo. Adicionalmente, el emisor implementa detección de **cables sueltos** mediante los pines LO+ y LO- del sensor AD8232: la función `leer_sensor()` verifica `if self.lo_plus.value() == 1 or self.lo_minus.value() == 1: return 0` antes de cualquier procesamiento difuso, y posteriormente en `procesar_fuzzy()` se realiza una doble verificación `if raw == 0 and (self.lo_plus.value() == 1 or self.lo_minus.value() == 1): self.debug_norm = 0; return 0`, garantizando que desconexiones accidentales de electrodos no generen lecturas espurias que podrían activar motores inesperadamente, una capa de seguridad completamente ausente en la simulación Python que asume sensores siempre funcionales.

5.5.2. Casos de Respuestas Bajas (<30%) - Parte 2: Frenado Dinámico y Control Diferencial

El segundo conjunto de casos de respuesta baja revela la inteligencia temporal del sistema difuso MicroPython en escenarios dinámicos. Durante **contracciones decrecientes rápidas** observables en las gráficas donde la entrada normalizada cae de 70% a 30% en aproximadamente 150-200 ms (3-4 ciclos a 20 Hz), el emisor detecta derivadas fuertemente negativas ($\Delta \approx -40\%$) que activan la función de membresía $\mu_{\Delta}[\text{'NB'}]=1.0$ (Negativo Big, definida como $\text{trapmf}(\Delta, -100, -50, -30, -15)$), modificando drásticamente las reglas de inferencia: en el primer ciclo con $\text{norm}=70\%$, $\Delta=-40\%$ la regla ($\mu_{\text{amp}}[\text{'A'}]=1.0$, $\mu_{\Delta}[\text{'NB'}]=1.0$) $\rightarrow$ 50% reduce la salida de los 75% esperados por amplitud sola a apenas 50%, y en ciclos subsecuentes con $\text{norm}=50\%$, $\Delta=-30\%$ la regla (M, NS) $\rightarrow$ 35% continúa atenuando hasta converger a $\text{norm}=30\%$, $\Delta \approx 0$ donde finalmente estabiliza en (B, Z) $\rightarrow$ 25%. Este comportamiento genera las **curvas exponenciales de decaimiento** visibles en las gráficas del plotter (imagen con señales descendentes suaves en lugar de caídas abruptas), previniendo que comandos de frenado brusco se traduzcan en desaceleraciones violentas del vehículo que podrían causar inestabilidad mecánica o volcaduras, mientras que Python al calcular instantáneamente sin memoria temporal produciría transiciones escalonadas 75% $\rightarrow$ 50% $\rightarrow$ 25% sin la suavización adaptativa. El caso final de respuesta baja ocurre durante **giros asimétricos sostenidos** evidenciados en múltiples registros de la consola del plotter como L_In:100, L_Out:100, R_In:0, R_Out:0 o L_In:44, L_Out:33, R_In:0, R_Out:0, donde el usuario mantiene contracción intensa en un brazo mientras el otro permanece completamente relajado: el canal activo genera salidas altas (33-100%) mientras el relajado produce 0%, creando diferencias mayores a 25% que el receptor detecta mediante $\text{diferencia} = \text{izq} - \text{der}$ y evalúa $\text{if abs(diferencia)} > 25$: $\text{fuerza_giro} = \max(\text{izq}, \text{der})$; $\text{pwm_giro} = \text{mapear_pwm}(\text{fuerza_giro}, 15, 100)$, ejecutando rotación diferencial donde si $\text{diferencia} > 0$ (izquierda domina) ejecuta $\text{motor_izq}(\text{pwm_giro}, \text{False})$; $\text{motor_der}(\text{pwm_giro}, \text{True})$ generando giro hacia la izquierda, o inversamente si $\text{diferencia} < 0$ (derecha domina) invierte las polaridades para girar a la derecha. Este control diferencial validado en las gráficas demuestra la correcta implementación de la matriz 5×5 de reglas difusas que evalúan independientemente cada canal sin interferencia cruzada, permitiendo maniobras complejas como giros en el lugar, curvas suaves, o

correcciones de trayectoria mediante ajustes sutiles de asimetría muscular, capacidades que aunque están implícitas en las superficies 3D de la simulación Python (donde planos diagonales representan movimientos asimétricos), solo se confirman funcionales mediante las pruebas experimentales en tiempo real capturadas por el plotter de Thonny que evidencian transiciones suaves, estabilidad sin oscilaciones, y respuesta proporcional tanto en magnitud como en dirección del comando diferencial.

5.6. PRUEBAS DE FUNCIONAMIENTO (VIDEO)

Liga de acceso al Google drive:

https://drive.google.com/file/d/1KOfn_6VZXHVJwPuTq4eD75WB5zRUTJQa/view?usp=sharing

https://drive.google.com/file/d/1KOfn_6VZXHVJwPuTq4eD75WB5zRUTJQa/view?usp=sharing

5.8. Código Fuente

5.8.1. Implementación de Firmware (Códigos Fuente)

El sistema consta de dos nodos ESP32 comunicados vía UDP/WiFi en una arquitectura Cliente-Servidor (Station-AP). A continuación, se presentan los códigos finales optimizados para el entorno Thonny IDE

```

# --- MOTOR DE INFERENCIA DIFUSA ---
def trimf(x, a, b, c):
    """Función de pertenencia Triangular"""
    if x <= a or x >= c: return 0.0
    if x == b: return 1.0
    if x < b: return (x - a) / (b - a)
    return (c - x) / (c - b)

def trapmf(x, a, b, c, d):
    """Función de pertenencia Trapezoidal"""
    if x <= a or x >= d: return 0.0
    if b <= x <= c: return 1.0
    if x < b: return (x - a) / (b - a)
    return (d - x) / (d - c)

class FuzzyEMG:
    def __init__(self, pin_adc, pin_lo_p, pin_lo_n):
        self.adc = ADC(Pin(pin_adc))
        self.adc.atten(ADC.ATTN_11DB)
        self.adc.width(ADC.WIDTH_12BIT)
        self.lo_plus = Pin(pin_lo_p, Pin.IN)
        self.lo_minus = Pin(pin_lo_n, Pin.IN)

        # Variables de estado
        self.min_val = 4095
        self.max_val = 0
        self.lectura_anterior = 0
        self.debug_norm = 0 # Para visualización

    def leer_sensor(self):
        # Seguridad: Retorna 0 si hay desconexión de electrodos
        if self.lo_plus.value() == 1 or self.lo_minus.value() == 1:
            return 0
        return self.adc.read()

    def calibrar(self):
        val = self.leer_sensor()
        if val < self.min_val: self.min_val = val
        if val > self.max_val: self.max_val = val
        if self.max_val == self.min_val: self.max_val = self.min_val + 100

```


5.8.2. Firmware del Emisor (Procesamiento Difuso)

Funciones: Adquisición ADC, Filtrado, Lógica Difusa (25 Reglas), Detección de desconexión (Leads-Off), Telemetría UDP

6.CONCLUSIONES

6.1. Valeria Abigail González Sada 1963075

El desarrollo del robot teleoperado mediante el sensor EMG AD8832 representó un avance significativo en la integración de biosensores con sistemas embebidos para aplicaciones de control inteligente. Logré establecer una comunicación efectiva entre el exoesqueleto —compuesto por el ESP32, el sensor EMG y una pila de 9V— y el vehículo receptor, permitiendo que los gestos musculares del brazo se tradujeran en movimientos direccionales como adelante, atrás, izquierda y derecha. La implementación de lógica difusa fue clave para interpretar las señales EMG con mayor precisión, simulando una dinámica de juego tipo fútbol que demostró la capacidad del sistema para responder de forma adaptativa y fluida.

Uno de los principales retos fue encontrar llantas que se adaptaran las reglas difusas del control difuso mediante el método del centroide usando señales trapezoidales y triangulares por medio de la librería skfuzzy.

En conjunto, el proyecto me permitió consolidar habilidades en integración electrónica, control por lógica difusa y adaptación mecánica, enfrentando desafíos reales de compatibilidad, precisión y respuesta. La experiencia de conectar el gesto humano con el movimiento robótico de forma inalámbrica y en tiempo real me confirmó el potencial de los sistemas EMG para aplicaciones de asistencia, deporte y rehabilitación. Este prototipo no solo cumplió con los objetivos técnicos planteados, sino que abrió nuevas posibilidades para escalar el diseño hacia entornos más complejos y colaborativos.

7. SUGERENCIAS FUTURAS

Una de las principales extensiones que considero valiosa es la incorporación de un sistema de retroalimentación háptica en el exoesqueleto, que permita al usuario sentir vibraciones o pulsos según la respuesta del robot. Esto no solo enriquecería la experiencia de teleoperación, sino que también facilitaría el entrenamiento muscular y la rehabilitación, al cerrar el ciclo sensorial entre intención y acción. Además, podría integrarse una pantalla OLED o indicadores LED para mostrar en tiempo real el estado del gesto detectado y la dirección ejecutada, mejorando la interacción humano-máquina.

Otra mejora clave sería la optimización del procesamiento de señales EMG mediante

técnicas de aprendizaje automático ligero, como redes neuronales convolucionales simplificadas o clasificadores SVM embebidos. Aunque la lógica difusa ha demostrado ser eficaz para interpretar gestos básicos, una capa de aprendizaje adaptativo permitiría personalizar el sistema según el perfil muscular de cada usuario, aumentando la precisión y reduciendo falsos positivos. Esto sería especialmente útil en aplicaciones clínicas o deportivas donde la variabilidad fisiológica es alta.

En cuanto al hardware, propongo migrar a una arquitectura modular con conectores rápidos y componentes intercambiables, lo que facilitaría el mantenimiento y la escalabilidad del sistema. Por ejemplo, el chasis del robot receptor podría rediseñarse con materiales más ligeros y resistentes, como PLA reforzado o acrílico cortado por láser, y las llantas podrían estandarizarse con adaptadores impresos en 3D para evitar futuras incompatibilidades como las que enfrenté al ajustar las de 4 mm. Esta modularidad permitiría probar diferentes configuraciones de motores, sensores y baterías sin rehacer el sistema completo.

Finalmente, veo potencial en extender el proyecto hacia entornos colaborativos multiagente, donde varios robots receptores respondan a gestos diferenciados de un mismo operador o de múltiples usuarios. Esto abriría posibilidades en simulaciones de juego, entrenamiento táctico o asistencia distribuida. Para ello, sería necesario implementar protocolos de comunicación más robustos como ESP-NOW o LoRa, y diseñar una interfaz de gestión centralizada que permita visualizar y coordinar los movimientos en tiempo real. Esta evolución convertiría el prototipo actual en una plataforma versátil para investigación, educación y desarrollo tecnológico inclusivo.

8. COMENTARIOS PERSONALES

8.1. Valeria Abigail González Sada 1963075

Como responsable de este proyecto, puedo decir que fue una experiencia enriquecedora en la que logré integrar conocimientos de electrónica, programación y control inteligente para transformar señales musculares en acciones robóticas reales. Ver cómo los gestos captados por el EMG se traducían en movimientos precisos del robot me permitió confirmar la utilidad de la lógica difusa y la comunicación WIFI -Wlan como herramientas confiables y adaptables. Más allá de la parte técnica, este trabajo me dejó la satisfacción de haber construido un sistema funcional que conecta directamente la

intención humana con la acción mecánica, abriendo posibilidades para aplicaciones en rehabilitación, asistencia y control inclusivo.

.9. ANEXOS

9.1. (A). Repartición de Tareas

1. Valeria Abigail González Sada 1963075

Me encargué de construir toda la parte mecánica de las correcciones, la parte electrónica donde se estuvieron realizando varias modificaciones por ejemplo en la etapa de potencia del microcontrolador del esp32 del receptor (carrito), además de estar realizando todas las pruebas y realizando las respectivas programaciones con los códigos para validar el comportamiento y simulación usando Python, Google colab y entorno de Thonny, además de los esquemáticos de la parte electrónica y toda la investigación y redacción del documento final.



9.2. (A) Pseudocódigo 1 – Ploteo de motores (Python)

PROGRAMA ControlDifusoEMG_Motores

```
// =====
// 1. DEFINICIÓN DE UNIVERSOS
// =====

DEFINIR Universo_EMG = rango [0, 4095] con paso 1
DEFINIR Universo_PWM = rango [0, 100] con paso 1

CREAR Antecedente EMG_Izquierdo con Universo_EMG
CREAR Antecedente EMG_Derecho con Universo_EMG
```

CREAR Consecuente Motor_Izquierdo con Universo_PWM

CREAR Consecuente Motor_Derecho con Universo_PWM

// =====

// 2. FUNCIONES DE MEMBRESÍA

// =====

// Entradas (EMG): Reposo, MuyBaja, Media, Alta, Hulk

PARA CADA sensor EN {EMG_Izquierdo, EMG_Derecho} HACER

DEFINIR sensor.Reposo = trapecio [0, 0, 800, 1200]

DEFINIR sensor.MuyBaja = triangulo [1000, 1500, 2000]

DEFINIR sensor.Media = triangulo [1800, 2500, 3200]

DEFINIR sensor.Alta = triangulo [3000, 3500, 4000]

DEFINIR sensor.Hulk = trapecio [3800, 4096, 4096, 4100]

FIN PARA

// Salidas (Motores): Paro, Lento, Crucero, Rapido, Turbo

PARA CADA motor EN {Motor_Izquierdo, Motor_Derecho} HACER

DEFINIR motor.Paro = trapecio [0, 0, 10, 20]

DEFINIR motor.Lento = triangulo [15, 30, 45]

DEFINIR motor.Crucero = triangulo [40, 55, 70]

DEFINIR motor.Rapido = triangulo [65, 80, 95]

DEFINIR motor.Turbo = trapecio [90, 100, 100, 100]

FIN PARA

// (Opcional) Visualizar funciones de membresía:

// LLAMAR a VerMembresias(EMG_Izquierdo)

// LLAMAR a VerMembresias(Motor_Izquierdo)

// =====

// 3. BASE DE REGLAS (25 REGLAS)

// =====

DEFINIR Lista_Reglas = vacía

```

// Niveles ordenados para EMG
EMG_Niveles_Izq = [Reposo, MuyBaja, Media, Alta, Hulk] de EMG_Izquierdo
EMG_Niveles_Der = [Reposo, MuyBaja, Media, Alta, Hulk] de EMG_Derecho

// Mapeo directo de nivel EMG → nivel PWM
PWM_Niveles = ["Paro", "Lento", "Crucero", "Rapido", "Turbo"]

CONTADOR idx = 1
PARA i DESDE 0 HASTA 4 HACER
    PARA j DESDE 0 HASTA 4 HACER
        // Motor izquierdo sigue EMG izquierdo en el mismo nivel
        OUT_I = Motor_Izquierdo[ PWM_Niveles[i] ]
        // Motor derecho sigue EMG derecho en el mismo nivel
        OUT_D = Motor_Derecho[ PWM_Niveles[j] ]

        REGLA R = SI (EMG_Niveles_Izq[i] AND EMG_Niveles_Der[j])
            ENTONCES (OUT_I, OUT_D)

    AÑADIR R A Lista_Reglas

    IMPRIMIR "Regla idx: Si Izq=names_emg[i] y Der=names_emg[j] -> " +
        "M_Izq=PWM_Niveles[i], M_Der=PWM_Niveles[j]"

    idx = idx + 1
FIN PARA
FIN PARA

// =====
// 4. SISTEMA DE CONTROL
// =====

CREAR Sistema_Control con Lista_Reglas
CREAR Simulacion ligada a Sistema_Control

```

```
// =====
```

```
// 5. PRUEBAS DE ESCRITORIO
```

```
// =====
```

```
IMPRIMIR "\n--- PRUEBA DE ESCRITORIO ---"
```

```
// Caso 1: Ambos medios → avanzar recto
```

```
ESTABLECER Simulacion.entrada.EMG_Izquierdo = 2500
```

```
ESTABLECER Simulacion.entrada.EMG_Derecho = 2500
```

```
COMPUTAR Simulacion
```

```
LEER pwm_i = Simulacion.salida.Motor_Izquierdo
```

```
LEER pwm_d = Simulacion.salida.Motor_Derecho
```

```
IMPRIMIR "Entrada(2500, 2500) -> Motor Izq: pwm_i%, Motor Der: pwm_d%"
```

```
// Caso 2: Izq muy alto, Der bajo → giro a la derecha
```

```
ESTABLECER Simulacion.entrada.EMG_Izquierdo = 3800
```

```
ESTABLECER Simulacion.entrada.EMG_Derecho = 1000
```

```
COMPUTAR Simulacion
```

```
LEER pwm_i = Simulacion.salida.Motor_Izquierdo
```

```
LEER pwm_d = Simulacion.salida.Motor_Derecho
```

```
IMPRIMIR "Entrada(3800, 1000) -> Motor Izq: pwm_i%, Motor Der: pwm_d%"
```

```
// =====
```

```
// 6. SUPERFICIES 3D (Barrido)
```

```
// =====
```

```
IMPRIMIR "\nGenerando superficie de control 3D..."
```

```
DEFINIR x_vals = 50 puntos equiespaciados entre 0 y 4096
```

```
DEFINIR y_vals = 50 puntos equiespaciados entre 0 y 4096
```

```
CREAR malla (X, Y) con x_vals, y_vals
```

```
CREAR matrices Z_izq y Z_der del tamaño de X llenas con 0
```

PARA i DESDE 0 HASTA 49 HACER

PARA j DESDE 0 HASTA 49 HACER

ESTABLECER Simulacion.entrada.EMG_Izquierdo = X[i, j]

ESTABLECER Simulacion.entrada.EMG_Derecho = Y[i, j]

COMPUTAR Simulacion

Z_izq[i, j] = Simulacion.salida.Motor_Izquierdo

Z_der[i, j] = Simulacion.salida.Motor_Derecho

FIN PARA

FIN PARA

// Dibujar superficies (titulos, ejes, colormaps)

LLAMAR DibujarSuperficie3D(X, Y, Z_izq,

 titulo="Superficie de Control: Motor Izquierdo",

 xlabel="EMG Izquierdo", ylabel="EMG Derecho",

 ylabel="Velocidad Motor Izq (%)", colormap="viridis")

LLAMAR DibujarSuperficie3D(X, Y, Z_der,

 titulo="Superficie de Control: Motor Derecho",

 xlabel="EMG Izquierdo", ylabel="EMG Derecho",

 ylabel="Velocidad Motor Der (%)", colormap="plasma")

FIN PROGRAMA

B. PSEUCÓDIGO 2 – graficos 2D EMG A PWM

PROGRAMA ControlDifuso_EMG_a_PWM_ConGraficas

// =====

// 1. DEFINICIÓN DE VARIABLES (Universos)

// =====

DEFINIR Universo_EMG = valores enteros de 0 a 4095 (paso 1)

DEFINIR Universo_PWM = valores enteros de 0 a 100 (paso 1)

CREAR Antecedente EMG_Izquierdo con Universo_EMG

CREAR Antecedente EMG_Derecho con Universo_EMG

CREAR Consecuente Motor_Izquierdo con Universo_PWM

CREAR Consecuente Motor_Derecho con Universo_PWM

// =====

// 2. FUNCIONES DE MEMBRESÍA

// =====

// Entradas (Sensores EMG): Reposo, MuyBaja, Media, Alta, Hulk

PARA sensor EN {EMG_Izquierdo, EMG_Derecho} HACER

DEFINIR sensor.Reposo = trapecio [0, 0, 800, 1200]

DEFINIR sensor.MuyBaja = triángulo [1000, 1500, 2000]

DEFINIR sensor.Media = triángulo [1800, 2500, 3200]

DEFINIR sensor.Alta = triángulo [3000, 3500, 4000]

DEFINIR sensor.Hulk = trapecio [3800, 4096, 4096, 4100]

FIN PARA

// Salidas (Motores PWM): Paro, Lento, Crucero, Rapido, Turbo

PARA motor EN {Motor_Izquierdo, Motor_Derecho} HACER

DEFINIR motor.Paro = trapecio [0, 0, 10, 20]

DEFINIR motor.Lento = triángulo [15, 30, 45]

DEFINIR motor.Crucero = triángulo [40, 55, 70]

DEFINIR motor.Rapido = triángulo [65, 80, 95]

DEFINIR motor.Turbo = trapecio [90, 100, 100, 100]

FIN PARA

// =====

// 3. BASE DE REGLAS (5x5 = 25 reglas)

// =====

INICIALIZAR lista REGLAS = []

EMG_Niveles_Izq = [EMG_Izquierdo.Reposo, EMG_Izquierdo.MuyBaja,

EMG_Izquierdo.Media, EMG_Izquierdo.Alta, EMG_Izquierdo.Hulk]

EMG_Niveles_Der = [EMG_Derecho.Reposo, EMG_Derecho.MuyBaja,
EMG_Derecho.Media, EMG_Derecho.Alta, EMG_Derecho.Hulk]

Nombres_Salida = ["Paro", "Lento", "Crucero", "Rapido", "Turbo"]

PARA i DESDE 0 HASTA 4 HACER

PARA j DESDE 0 HASTA 4 HACER

OUT_I = Motor_Izquierdo[Nombres_Salida[i]] // Mapeo directo: EMG_i →
PWM_i
OUT_D = Motor_Derecho[Nombres_Salida[j]] // Mapeo directo: EMG_j →
PWM_j

REGLA r = SI (EMG_Niveles_Izq[i] AND EMG_Niveles_Der[j])
ENTONCES (OUT_I, OUT_D)

AGREGAR r A REGLAS

FIN PARA

FIN PARA

// Construir sistema y simulador

CREAR Sistema_Control con REGLAS

CREAR Simulacion asociada a Sistema_Control

// =====

// 4. CONFIGURAR LA PRUEBA (Simulación puntual)

// =====

// Valores ajustables por el usuario para reproducir la gráfica deseada

DEFINIR val_izq = 3500 // ejemplo: fuerza alta en izquierdo

DEFINIR val_der = 0 // ejemplo: reposo en derecho

ESTABLECER Simulacion.entrada.EMG_Izquierdo = val_izq

```
ESTABLECER Simulacion.entrada.EMG_Derecho = val_der
```

```
EJECUTAR Simulacion.compute()
```

```
LEER pwm_izq = Simulacion.salida.Motor_Izquierdo // porcentaje 0-100
```

```
LEER pwm_der = Simulacion.salida.Motor_Derecho // porcentaje 0-100
```

```
MOSTRAR "Resultados -> MI: pwm_izq %, MD: pwm_der %"
```

9.2 ©. CÓDIGO FINAL DEL EMISOR

```
import network
```

```
import socket
```

```
import time
```

```
from machine import Pin, ADC
```

```
#
```

```
=====
```

```
# 1. CONFIGURACIÓN WIFI (CLIENTE)
```

```
#
```

```
=====
```

```
SSID_CARRO = "EMG_CAR"
```

```
PASS_CARRO = "12345678"
```

```
DEST_IP = "192.168.4.1"
```

```
DEST_PORT = 5005
```

```
led = Pin(2, Pin.OUT)
```

```
led.value(0) # Empezamos apagado
```

```
wlan = network.WLAN(network.STA_IF)
```

```
wlan.active(True)
```

```

wlan.connect(SSID_CARRO, PASS_CARRO)

print("--- INICIANDO EMISOR ---")
print("Conectando al WiFi del Carro...")
timeout = 0
while not wlan.isconnected() and timeout < 10:
    led.value(1); time.sleep(0.2); led.value(0); time.sleep(0.2)
    timeout += 1
    print(".", end="")

print("") # Salto de linea
if wlan.isconnected():
    print(f"; CONECTADO! IP Emisor: {wlan.ifconfig()[0]}")
    led.value(1) # Led fijo = Conectado
else:
    print("ADVERTENCIA: No se pudo conectar. Revisar si el Carro está encendido.")
    # No detenemos el código para permitir probar la lectura de sensores offline

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

#
=====

=====

# 2. MOTOR DE LÓGICA DIFUSA (CLASE OPTIMIZADA)
#
=====

=====

def trimf(x, a, b, c):
    if x <= a or x >= c: return 0.0
    if x == b: return 1.0
    if x < b: return (x - a) / (b - a)
    return (c - x) / (c - b)

```

```
def trapmf(x, a, b, c, d):
```

```
    if x <= a or x >= d: return 0.0
```

```
    if b <= x <= c: return 1.0
```

```
    if x < b: return (x - a) / (b - a)
```

```
    return (d - x) / (d - c)
```

```
class FuzzyEMG:
```

```
    def __init__(self, pin_adc, pin_lo_p, pin_lo_n, nombre):
```

```
        # Configuración Sensor
```

```
        self.adc = ADC(Pin(pin_adc))
```

```
        self.adc.atten(ADC.ATTN_11DB)
```

```
        self.adc.width(ADC.WIDTH_12BIT)
```

```
        # Pines Leads Off (Desconexión)
```

```
        self.lo_plus = Pin(pin_lo_p, Pin.IN)
```

```
        self.lo_minus = Pin(pin_lo_n, Pin.IN)
```

```
        self.nombre = nombre
```

```
        # Variables Fuzzy
```

```
        self.min_val = 4095
```

```
        self.max_val = 0
```

```
        self.lectura_anterior = 0
```

```
        self.debug_norm = 0 # Variable nueva para visualizar en Thonny
```

```
    def leer_sensor(self):
```

```
        # Si detecta cable suelto, retorna 0
```

```
        if self.lo_plus.value() == 1 or self.lo_minus.value() == 1:
```

```
            return 0
```

```
        return self.adc.read()
```

```
    def calibrar(self):
```

```
        val = self.leer_sensor()
```

```

if val < self.min_val: self.min_val = val
if val > self.max_val: self.max_val = val
if self.max_val == self.min_val: self.max_val = self.min_val + 100

```

```
def procesar_fuzzy(self):
```

```
    # 1. Leer y Normalizar (0 a 100)
```

```
    raw = self.leer_sensor()
```

```
    # Seguridad desconexión
```

```
    if raw == 0 and (self.lo_plus.value() == 1 or self.lo_minus.value() == 1):
```

```
        self.debug_norm = 0
```

```
        return 0
```

```
    norm = ((raw - self.min_val) / (self.max_val - self.min_val)) * 100
```

```
    norm = max(0, min(100, norm))
```

```
    self.debug_norm = norm # Guardamos para el print del plotter
```

```
    # 2. Calcular Derivada (Velocidad de cambio)
```

```
    delta = norm - self.lectura_anterior
```

```
    self.lectura_anterior = norm
```

```
    # 3. Fuzzificar Entradas
```

```
    mu_amp = {
```

```
        'MB': trapmf(norm, -1, 0, 10, 25),
```

```
        'B': trimf(norm, 10, 30, 50),
```

```
        'M': trimf(norm, 30, 50, 70),
```

```
        'A': trimf(norm, 50, 70, 90),
```

```
        'MA': trapmf(norm, 75, 90, 100, 101)
```

```
    }
```

```
    mu_delta = {
```

```
        'NB': trapmf(delta, -100, -50, -30, -15),
```

```
        'NS': trimf(delta, -30, -15, 0),
```

```
        'Z': trimf(delta, -15, 0, 15),
```

```
'PS': trimf(delta, 0, 15, 30),
'PB': trapmf(delta, 15, 30, 50, 100)
}
```

4. Inferencia (25 Reglas Compactas)

```
reglas = []

# Fila MB (Muy Bajo)
reglas.append((min(mu_amp['MB'], mu_delta['NB']), 0));
reglas.append((min(mu_amp['MB'], mu_delta['NS']), 0))
reglas.append((min(mu_amp['MB'], mu_delta['Z']), 0));
reglas.append((min(mu_amp['MB'], mu_delta['PS']), 10))
reglas.append((min(mu_amp['MB'], mu_delta['PB']), 20))

# Fila B (Bajo)
reglas.append((min(mu_amp['B'], mu_delta['NB']), 0));
reglas.append((min(mu_amp['B'], mu_delta['NS']), 10))
reglas.append((min(mu_amp['B'], mu_delta['Z']), 25));
reglas.append((min(mu_amp['B'], mu_delta['PS']), 35))
reglas.append((min(mu_amp['B'], mu_delta['PB']), 45))

# Fila M (Medio)
reglas.append((min(mu_amp['M'], mu_delta['NB']), 25));
reglas.append((min(mu_amp['M'], mu_delta['NS']), 35))
reglas.append((min(mu_amp['M'], mu_delta['Z']), 50));
reglas.append((min(mu_amp['M'], mu_delta['PS']), 60))
reglas.append((min(mu_amp['M'], mu_delta['PB']), 70))

# Fila A (Alto)
reglas.append((min(mu_amp['A'], mu_delta['NB']), 50));
reglas.append((min(mu_amp['A'], mu_delta['NS']), 60))
reglas.append((min(mu_amp['A'], mu_delta['Z']), 75));
reglas.append((min(mu_amp['A'], mu_delta['PS']), 85))
reglas.append((min(mu_amp['A'], mu_delta['PB']), 90))

# Fila MA (Muy Alto)
reglas.append((min(mu_amp['MA'], mu_delta['NB']), 70));
reglas.append((min(mu_amp['MA'], mu_delta['NS']), 80))
```

```

reglas.append((min(mu_amp['MA'], mu_delta['Z']), 100));
reglas.append((min(mu_amp['MA'], mu_delta['PS']), 100))
reglas.append((min(mu_amp['MA'], mu_delta['PB']), 100))

```

```

# 5. Defuzzificación (Centroide)

```

```

num = 0; den = 0

```

```

for disparo, val_salida in reglas:

```

```

    num += disparo * val_salida

```

```

    den += disparo

```

```

if den == 0: return 0

```

```

return int(num / den)

```

```

#

```

```

=====
=====

```

```

# 3. OBJETOS Y CALIBRACIÓN

```

```

#

```

```

=====
=====

```

```

brazo_izq = FuzzyEMG(36, 4, 14, "Izquierdo") # ADC 36

```

```

brazo_der = FuzzyEMG(39, 16, 17, "Derecho") # ADC 39

```

```

print("--- CALIBRACIÓN (5s) ---")

```

```

print("INSTRUCCIÓN: Contrae y relaja AMBOS brazos fuertemente.")

```

```

start_time = time.time()

```

```

while (time.time() - start_time) < 5:

```

```

    led.value(not led.value()) # Parpadeo rápido

```

```

    brazo_izq.calibrar()

```

```

    brazo_der.calibrar()

```

```

    time.sleep_ms(10)

```

```

led.value(1) # Listo

```



```

print(f'Rango Izq: {brazo_izq.min_val}-{brazo_izq.max_val}')
print(f'Rango Der: {brazo_der.min_val}-{brazo_der.max_val}')
print("--- TRANSMITIENDO ---")
print("Leyenda Plotter: L_In=Entrada Izq, L_Out=Salida Izq, R_In=Entrada Der,
R_Out=Salida Der")

#
=====

=====

# 4. BUCLE PRINCIPAL (CON ESCAPE)
#
=====

=====

try:
    while True:
        # 1. Procesamiento Difuso
        vel_izq = brazo_izq.procesar_fuzzy()
        vel_der = brazo_der.procesar_fuzzy()

        # 2. VISUALIZACIÓN MEJORADA (Para Thonny Plotter)
        # Mostramos Entrada (debug_norm) vs Salida (vel) para ver la relación
        # Formato: "Variable:Valor,Variable:Valor..."
        # L_In = Left Input (Fuerza tuya), L_Out = Left Output (Lo que va al motor)

        print(f'L_In: {int(brazo_izq.debug_norm)},L_Out:{vel_izq},R_In: {int(brazo_der.debug
_norm)},R_Out:{vel_der}')

        # 3. Enviar Datos
        mensaje = "{} , {}".format(vel_izq, vel_der)

        try:
            sock.sendto(mensaje.encode(), (DEST_IP, DEST_PORT))
        except:

```

pass # Ignoramos errores de red puntuales

time.sleep_ms(50) # 20 Hz

except KeyboardInterrupt:

print("\n--- DETENIDO POR USUARIO (Ctrl+C) ---")

led.value(0)

sock.close()

9.2 (D) – CÓDIGO FINAL DEL RECEPTOR

import network

import socket

import machine

import time

from machine import Pin, PWM

#

=====

1. CONFIGURACIÓN HARDWARE

#

=====

led = Pin(2, Pin.OUT)

Motores (Ajusta pines si es necesario)

m1_in1 = PWM(Pin(19), freq=1000)

m1_in2 = PWM(Pin(18), freq=1000)

m2_in1 = PWM(Pin(23), freq=1000)

m2_in2 = PWM(Pin(22), freq=1000)

AJUSTE DE PISO (Potencia mínima)

MIN_PWM = 450

#

```
=====
=====
```

2. FUNCIONES DE MOTORES

#

```
=====
=====
```

def parar():

```
    """Detiene todos los motores inmediatamente"""
```

```
    m1_in1.duty(0); m1_in2.duty(0)
```

```
    m2_in1.duty(0); m2_in2.duty(0)
```

def motor_izq(pwm, adelante):

```
    if adelante:
```

```
        m1_in1.duty(pwm); m1_in2.duty(0)
```

```
    else:
```

```
        m1_in1.duty(0); m1_in2.duty(pwm)
```

def motor_der(pwm, adelante):

```
    if adelante:
```

```
        m2_in1.duty(pwm); m2_in2.duty(0)
```

```
    else:
```

```
        m2_in1.duty(0); m2_in2.duty(pwm)
```

def mapear_pwm(valor_entrada, in_min, in_max):

```
    """Escala un rango de entrada (ej 15-40) al rango PWM del motor"""
```

```
    if valor_entrada < in_min: return MIN_PWM
```

```
    if valor_entrada > in_max: return 1023
```

```
    rango_in = in_max - in_min
```

```
    rango_out = 1023 - MIN_PWM
```

```
    pwm = int(((valor_entrada - in_min) * rango_out / rango_in) + MIN_PWM)
```

```
return pwm
```

```
#
```

```
=====
```

3. LÓGICA DE MOVIMIENTO (ZONAS)

```
#
```

```
=====
```

```
def procesar_movimiento(izq, der):
```

```
    promedio = (izq + der) / 2
```

```
    diferencia = izq - der
```

```
# --- ZONA MUERTA (Reposo) ---
```

```
if izq < 12 and der < 12:
```

```
    parar()
```

```
    led.value(0)
```

```
    return
```

```
# --- PRIORIDAD 1: GIROS (Asimetría) ---
```

```
if abs(diferencia) > 25:
```

```
    fuerza_giro = max(izq, der)
```

```
    pwm_giro = mapear_pwm(fuerza_giro, 15, 100)
```

```
if diferencia > 0: # Izquierda gana -> Giro Izq
```

```
    motor_izq(pwm_giro, False)
```

```
    motor_der(pwm_giro, True)
```

```
else:          # Derecha gana -> Giro Der
```

```
    motor_izq(pwm_giro, True)
```

```
    motor_der(pwm_giro, False)
```

```
led.value(1)
```

```
return
```

```
# --- PRIORIDAD 2: AVANCE / REVERSA ---
```

```
# REVERSA (Suave 15-40%)
```

```
if 15 <= promedio <= 40:
```

```
    pwm = mapear_pwm(promedio, 15, 40)
```

```
    motor_izq(pwm, False)
```

```
    motor_der(pwm, False)
```

```
    # Parpadeo rápido
```

```
    if (time.ticks_ms() % 200) < 100: led.value(1)
```

```
    else: led.value(0)
```

```
# ZONA SEGURIDAD (Hueco 41-49%)
```

```
elif 40 < promedio < 50:
```

```
    parar()
```

```
    led.value(0)
```

```
# ADELANTE (Fuerte 50-100%)
```

```
elif promedio >= 50:
```

```
    pwm = mapear_pwm(promedio, 50, 100)
```

```
    motor_izq(pwm, True)
```

```
    motor_der(pwm, True)
```

```
    led.value(1)
```

```
#
```

```
=====
```

```
=====
```

```
# 4. LOOP PRINCIPAL CON ESCAPE (Ctrl+C)
```

```
#
```

```
=====
```

```
=====
```

```
# Configuración WiFi
```

```
AP_SSID = "EMG_CAR"
```

```
AP_PASS = "12345678"
```

```
UDP_PORT = 5005
```

try:

Inicio WiFi

ap = network.WLAN(network.AP_IF)

ap.active(True)

ap.config(essid=AP_SSID, password=AP_PASS, authmode=3)

Inicio Socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

sock.bind(("0.0.0.0", UDP_PORT))

sock.settimeout(0.1)

print("--- SISTEMA LISTO ---")

print("Para salir de forma segura, presiona Ctrl+C en Thonny")

parar()

BUCLE INFINITO

while True:

try:

data, addr = sock.recvfrom(1024)

mensaje = data.decode().strip()

partes = mensaje.split(',')

if len(partes) == 2:

f_izq = int(partes[0])

f_der = int(partes[1])

procesar_movimiento(f_izq, f_der)

except OSError:

parar() # Timeout: parar por seguridad

except ValueError:

pass

except KeyboardInterrupt:

```
# --- AQUÍ ENTRA CUANDO PRESIONAS CTRL+C ---
parar()
led.value(0)
print("\n!!! PROGRAMA DETENIDO POR USUARIO !!!")
print("Motores apagados correctamente.")
```

9.2. PSEUCÓDIGO CODIGO FINAL DEL EMISOR

1. CONFIGURACIÓN INICIAL DE RED Y HARDWARE

```
DEFINIR configuración de red:
    nombre_red_destino = "EMG_CAR"
    contraseña = "12345678"
    IP_carro = "192.168.4.1"
    puerto_destino = 5005

CONFIGURAR led en pin 2 como salida
APAGAR led

INICIAR modo WiFi cliente
ACTIVAR WiFi
CONECTAR a red del carro

MOSTRAR "Conectando al WiFi del Carro..."

timeout_contador = 0
MIENTRAS NO conectado Y timeout_contador < 10:
    ENCENDER led, ESPERAR 0.2s
    APAGAR led, ESPERAR 0.2s
    INCREMENTAR timeout_contador
    MOSTRAR punto de progreso

SI conectado exitosamente:
    MOSTRAR ";CONECTADO! IP Emisor: [IP_asignada]"
    ENCENDER led (fijo)
SINO:
    MOSTRAR "ADVERTENCIA: No se pudo conectar"
    // Continúa para pruebas offline

CREAR socket UDP
```

2. FUNCIONES DE MEMBRESÍA DIFUSA

Función: trimf(x, a, b, c)

```
// Función triangular de membresía
SI  $x \leq a$  O  $x \geq c$ :
    RETORNAR 0.0
SI  $x == b$ :
```

```

    RETORNAR 1.0
SI x < b:
    RETORNAR (x - a) / (b - a)
SINO:
    RETORNAR (c - x) / (c - b)

```

Función: trapmf(x, a, b, c, d)

```

// Función trapezoidal de membresía
SI x ≤ a O x ≥ d:
    RETORNAR 0.0
SI b ≤ x ≤ c:
    RETORNAR 1.0
SI x < b:
    RETORNAR (x - a) / (b - a)
SINO:
    RETORNAR (d - x) / (d - c)

```

3. CLASE DE PROCESAMIENTO FUZZY EMG

Clase: FuzzyEMG

Constructor: init(pin_adc, pin_leads_off_plus, pin_leads_off_minus, nombre)

```

CONFIGURAR conversor analógico-digital:
    ASIGNAR pin ADC
    ESTABLECER atenuación a 11dB (rango 0-3.3V)
    ESTABLECER resolución a 12 bits (0-4095)

CONFIGURAR pines de detección de desconexión:
    pin_leads_off_plus como entrada
    pin_leads_off_minus como entrada

INICIALIZAR variables:
    nombre_sensor = nombre
    valor_mínimo_calibración = 4095
    valor_máximo_calibración = 0
    lectura_anterior = 0
    valor_normalizado_debug = 0

```

Método: leer_sensor()

```

SI pin_leads_off_plus == 1 O pin_leads_off_minus == 1:
    // Sensor desconectado
    RETORNAR 0

RETORNAR lectura del ADC

```

Método: calibrar()

```

valor = leer_sensor()

SI valor < valor_mínimo_calibración:
    valor_mínimo_calibración = valor

```



```
SI valor > valor_máximo_calibración:
    valor_máximo_calibración = valor
```

```
SI valor_máximo == valor_mínimo:
    valor_máximo_calibración = valor_mínimo_calibración + 100
```

Método: procesar_fuzzy()

```
// ===== PASO 1: LECTURA Y NORMALIZACIÓN =====
valor_crudo = leer_sensor()

// Verificar desconexión
SI valor_crudo == 0 Y (pin_leads_off_plus == 1 O pin_leads_off_minus
== 1):
    valor_normalizado_debug = 0
    RETORNAR 0

// Normalizar a rango 0-100
valor_normalizado = ((valor_crudo - valor_mínimo) / (valor_máximo -
valor_mínimo)) × 100
valor_normalizado = limitar(valor_normalizado, 0, 100)
valor_normalizado_debug = valor_normalizado

// ===== PASO 2: CALCULAR DERIVADA (Velocidad de cambio) =====
delta = valor_normalizado - lectura_anterior
lectura_anterior = valor_normalizado

// ===== PASO 3: FUZZIFICACIÓN DE ENTRADAS =====

// Conjuntos difusos para AMPLITUD (0-100%)
mu_amplitud = {
    'Muy Bajo': trapmf(valor_normalizado, -1, 0, 10, 25),
    'Bajo':      trimf(valor_normalizado, 10, 30, 50),
    'Medio':     trimf(valor_normalizado, 30, 50, 70),
    'Alto':      trimf(valor_normalizado, 50, 70, 90),
    'Muy Alto':  trapmf(valor_normalizado, 75, 90, 100, 101)
}

// Conjuntos difusos para DELTA (Tasa de cambio)
mu_delta = {
    'Negativo Grande': trapmf(delta, -100, -50, -30, -15),
    'Negativo Pequeño': trimf(delta, -30, -15, 0),
    'Cero':             trimf(delta, -15, 0, 15),
    'Positivo Pequeño': trimf(delta, 0, 15, 30),
    'Positivo Grande':  trapmf(delta, 15, 30, 50, 100)
}

// ===== PASO 4: INFERENCIA (25 REGLAS) =====

CREAR lista_reglas vacía

// Matriz de reglas 5×5 (Amplitud × Delta)
// Formato: (grado_activación, valor_salida_consecuente)

// FILA: Amplitud MUY BAJA
```

```

AGREGAR (min(mu_amplitud['MB'], mu_delta['NB']), 0)
AGREGAR (min(mu_amplitud['MB'], mu_delta['NS']), 0)
AGREGAR (min(mu_amplitud['MB'], mu_delta['Z']), 0)
AGREGAR (min(mu_amplitud['MB'], mu_delta['PS']), 10)
AGREGAR (min(mu_amplitud['MB'], mu_delta['PB']), 20)

// FILA: Amplitud BAJA
AGREGAR (min(mu_amplitud['B'], mu_delta['NB']), 0)
AGREGAR (min(mu_amplitud['B'], mu_delta['NS']), 10)
AGREGAR (min(mu_amplitud['B'], mu_delta['Z']), 25)
AGREGAR (min(mu_amplitud['B'], mu_delta['PS']), 35)
AGREGAR (min(mu_amplitud['B'], mu_delta['PB']), 45)

// FILA: Amplitud MEDIA
AGREGAR (min(mu_amplitud['M'], mu_delta['NB']), 25)
AGREGAR (min(mu_amplitud['M'], mu_delta['NS']), 35)
AGREGAR (min(mu_amplitud['M'], mu_delta['Z']), 50)
AGREGAR (min(mu_amplitud['M'], mu_delta['PS']), 60)
AGREGAR (min(mu_amplitud['M'], mu_delta['PB']), 70)

// FILA: Amplitud ALTA
AGREGAR (min(mu_amplitud['A'], mu_delta['NB']), 50)
AGREGAR (min(mu_amplitud['A'], mu_delta['NS']), 60)
AGREGAR (min(mu_amplitud['A'], mu_delta['Z']), 75)
AGREGAR (min(mu_amplitud['A'], mu_delta['PS']), 85)
AGREGAR (min(mu_amplitud['A'], mu_delta['PB']), 90)

// FILA: Amplitud MUY ALTA
AGREGAR (min(mu_amplitud['MA'], mu_delta['NB']), 70)
AGREGAR (min(mu_amplitud['MA'], mu_delta['NS']), 80)
AGREGAR (min(mu_amplitud['MA'], mu_delta['Z']), 100)
AGREGAR (min(mu_amplitud['MA'], mu_delta['PS']), 100)
AGREGAR (min(mu_amplitud['MA'], mu_delta['PB']), 100)

// ===== PASO 5: DEFUZZIFICACIÓN (Método del Centroide) =====

numerador = 0
denominador = 0

PARA CADA (activación, valor_salida) EN lista_reglas:
    numerador = numerador + (activación × valor_salida)
    denominador = denominador + activación

SI denominador == 0:
    RETORNAR 0

salida_crisp = numerador / denominador
RETORNAR salida_crisp como entero

```

4. INICIALIZACIÓN DE SENSORES Y CALIBRACIÓN

```

// Crear objetos para ambos brazos
sensor_brazo_izquierdo = FuzzyEMG(pin_adc=36, pin_lo_p=4, pin_lo_n=14,
nombre="Izquierdo")
sensor_brazo_derecho = FuzzyEMG(pin_adc=39, pin_lo_p=16, pin_lo_n=17,
nombre="Derecho")

```

```

MOSTRAR "--- CALIBRACIÓN (5 segundos) ---"
MOSTRAR "INSTRUCCIÓN: Contrae y relaja AMBOS brazos fuertemente"

tiempo_inicio = obtener_tiempo_actual()

MIENTRAS (tiempo_actual - tiempo_inicio) < 5 segundos:
    ALTERNAR estado del led (parpadeo rápido)
    sensor_brazo_izquierdo.calibrar()
    sensor_brazo_derecho.calibrar()
    ESPERAR 10 milisegundos

ENCENDER led (fijo = calibración completa)

MOSTRAR "Rango Izquierdo: [mínimo]-[máximo]"
MOSTRAR "Rango Derecho: [mínimo]-[máximo]"
MOSTRAR "--- TRANSMITIENDO ---"
MOSTRAR "Leyenda: L_In=Entrada Izq, L_Out=Salida Izq, R_In=Entrada
Der, R_Out=Salida Der"

```

5. BUCLE PRINCIPAL DE TRANSMISIÓN

```

INTENTAR:
    MIENTRAS verdadero:
        // --- PASO 1: Procesamiento Difuso ---
        velocidad_izquierda = sensor_brazo_izquierdo.procesar_fuzzy()
        velocidad_derecha = sensor_brazo_derecho.procesar_fuzzy()

        // --- PASO 2: Visualización para Plotter de Thonny ---
        entrada_izq = sensor_brazo_izquierdo.valor_normalizado_debug
        entrada_der = sensor_brazo_derecho.valor_normalizado_debug

        MOSTRAR
        "L_In:[entrada_izq],L_Out:[velocidad_izquierda],R_In:[entrada_der],R_O
ut:[velocidad_derecha]"

        // --- PASO 3: Preparar y Enviar Mensaje ---
        mensaje = "[velocidad_izquierda],[velocidad_derecha]"

        INTENTAR:
            ENVIAR mensaje vía UDP a (IP_carro, puerto_destino)
        CAPTURAR error:
            IGNORAR // Error de red puntual

        ESPERAR 50 milisegundos // Frecuencia de 20 Hz

CAPTURAR interrupción de teclado (Ctrl+C):
    MOSTRAR "--- DETENIDO POR USUARIO ---"
    APAGAR led
    CERRAR socket

```

9.2 (E)- PSEUCÓDIGO DE CÓDIGO FINAL DEL RECEPTOR

1. CONFIGURACIÓN INICIAL DEL HARDWARE

CONFIGURAR led en pin 2 como salida digital

CONFIGURAR motores con señales PWM:

```
motor1_entrada1 = PWM en pin 19 con frecuencia 1000 Hz
motor1_entrada2 = PWM en pin 18 con frecuencia 1000 Hz
motor2_entrada1 = PWM en pin 23 con frecuencia 1000 Hz
motor2_entrada2 = PWM en pin 22 con frecuencia 1000 Hz
```

DEFINIR constante:

```
PWM_MÍNIMO = 450 // Potencia mínima para que los motores
arranquen
```

2. FUNCIONES DE CONTROL DE MOTORES

Función: parar()

```
// Detiene todos los motores inmediatamente
ESTABLECER motor1_entrada1.duty = 0
ESTABLECER motor1_entrada2.duty = 0
ESTABLECER motor2_entrada1.duty = 0
ESTABLECER motor2_entrada2.duty = 0
```

Función: motor_izquierdo(potencia_pwm, dirección_adelante)

```
SI dirección_adelante ES verdadero:
    // Motor gira hacia adelante
    ESTABLECER motor1_entrada1.duty = potencia_pwm
    ESTABLECER motor1_entrada2.duty = 0
SINO:
    // Motor gira hacia atrás
    ESTABLECER motor1_entrada1.duty = 0
    ESTABLECER motor1_entrada2.duty = potencia_pwm
```

Función: motor_derecho(potencia_pwm, dirección_adelante)

```
SI dirección_adelante ES verdadero:
    // Motor gira hacia adelante
    ESTABLECER motor2_entrada1.duty = potencia_pwm
    ESTABLECER motor2_entrada2.duty = 0
SINO:
    // Motor gira hacia atrás
    ESTABLECER motor2_entrada1.duty = 0
    ESTABLECER motor2_entrada2.duty = potencia_pwm
```

Función: mapear_pwm(valor, mínimo_entrada, máximo_entrada)

```
// Escala un rango de entrada al rango PWM del motor (PWM_MÍNIMO a
1023)

// Protección de límites inferiores
SI valor < mínimo_entrada:
    RETORNAR PWM_MÍNIMO

// Protección de límites superiores
SI valor > máximo_entrada:
```

RETORNAR 1023

```
// Interpolación lineal
rango_entrada = máximo_entrada - mínimo_entrada
rango_salida = 1023 - PWM_MÍNIMO

pwm_escalado = ((valor - mínimo_entrada) × rango_salida /
rango_entrada) + PWM_MÍNIMO

RETORNAR pwm_escalado como entero
```

3. LÓGICA DE PROCESAMIENTO DE MOVIMIENTO

Función: procesar_movimiento(señal_izquierda, señal_derecha)

```
// Calcular parámetros de decisión
promedio = (señal_izquierda + señal_derecha) / 2
diferencia = señal_izquierda - señal_derecha

// =====
// ZONA 1: REPOSO (Zona Muerta)
// =====
SI señal_izquierda < 12 Y señal_derecha < 12:
    parar()
    APAGAR led
    SALIR de función

// =====
// ZONA 2: GIROS (Prioridad Alta)
// =====
// Se activa cuando hay asimetría significativa entre brazos

SI valor_absoluto(diferencia) > 25:
    fuerza_giro = máximo(señal_izquierda, señal_derecha)
    pwm_giro = mapear_pwm(fuerza_giro, 15, 100)

    SI diferencia > 0:
        // Señal izquierda domina → Giro a la IZQUIERDA
        // Motor izquierdo retrocede, motor derecho avanza
        motor_izquierdo(pwm_giro, ATRÁS)
        motor_derecho(pwm_giro, ADELANTE)
    SINO:
        // Señal derecha domina → Giro a la DERECHA
        // Motor izquierdo avanza, motor derecho retrocede
        motor_izquierdo(pwm_giro, ADELANTE)
        motor_derecho(pwm_giro, ATRÁS)

    ENCENDER led
    SALIR de función

// =====
// ZONA 3: MOVIMIENTO LINEAL
// =====
```

```
// --- REVERSA (Movimiento suave hacia atrás) ---
SI 15 ≤ promedio ≤ 40:
    pwm = mapear_pwm(promedio, 15, 40)
    motor_izquierdo(pwm, ATRÁS)
    motor_derecho(pwm, ATRÁS)

    // Indicación visual: parpadeo rápido
    tiempo_actual = obtener_tiempo_en_milisegundos()
    SI (tiempo_actual módulo 200) < 100:
        ENCENDER led
    SINO:
        APAGAR led

// --- ZONA DE SEGURIDAD (Hueco de transición) ---
SINO SI 40 < promedio < 50:
    // Zona neutral para evitar cambios bruscos
    parar()
    APAGAR led

// --- ADELANTE (Movimiento fuerte hacia adelante) ---
SINO SI promedio ≥ 50:
    pwm = mapear_pwm(promedio, 50, 100)
    motor_izquierdo(pwm, ADELANTE)
    motor_derecho(pwm, ADELANTE)
    ENCENDER led
```

4. CONFIGURACIÓN DE RED WIFI

```
DEFINIR parámetros de red:
    nombre_red = "EMG_CAR"
    contraseña_red = "12345678"
    puerto_UDP = 5005
```

5. PROGRAMA PRINCIPAL CON MANEJO DE ERRORES

```
INTENTAR:
    // =====
    // INICIALIZACIÓN DE RED
    // =====

    CREAR punto_acceso WiFi en modo AP (Access Point)
    ACTIVAR punto_acceso
    CONFIGURAR punto_acceso:
        SSID = nombre_red
        contraseña = contraseña_red
        modo_autenticación = 3 // WPA2

    // =====
    // INICIALIZACIÓN DE SOCKET UDP
    // =====

    CREAR socket UDP
    VINCULAR socket a:
        dirección = "0.0.0.0" // Escuchar en todas las interfaces
        puerto = puerto_UDP
```

```

ESTABLECER timeout del socket = 0.1 segundos // 100ms

// =====
// CONFIRMACIÓN DE INICIO
// =====

MOSTRAR "--- SISTEMA LISTO ---"
MOSTRAR "Para salir de forma segura, presiona Ctrl+C en Thonny"
parar() // Asegurar que motores están apagados

// =====
// BUCLE PRINCIPAL INFINITO
// =====

MIENTRAS verdadero:
    INTENTAR:
        // Esperar datos UDP (bloqueo con timeout)
        (datos, dirección_origen) =
socket.recibir(tamaño_máximo=1024)

        // Decodificar mensaje
        mensaje = decodificar(datos como texto UTF-8)
        mensaje = eliminar_espacios_blanco(mensaje)

        // Parsear mensaje formato: "valor_izq,valor_der"
        partes = dividir(mensaje, separador=',')

        SI cantidad_de_partes == 2:
            fuerza_izquierda = convertir_a_entero(partes[0])
            fuerza_derecha = convertir_a_entero(partes[1])

            // Procesar y ejecutar movimiento
            procesar_movimiento(fuerza_izquierda, fuerza_derecha)

        CAPTURAR error_de_timeout (OSError):
            // No se recibió datos en 100ms → Medida de seguridad
            parar()

        CAPTURAR error_de_conversión (ValueError):
            // Datos recibidos no son numéricos válidos
            IGNORAR // Continuar esperando próximo mensaje

// =====
// MANEJO DE INTERRUPCIÓN DEL USUARIO
// =====

CAPTURAR interrupción_de_teclado (Ctrl+C):
    // Procedimiento de apagado seguro
    parar()
    APAGAR led
    MOSTRAR "\n!!! PROGRAMA DETENIDO POR USUARIO !!!"
    MOSTRAR "Motores apagados correctamente."

```

10. REFERENCIAS DE APOYO

- Ali, A. R., Ramadan, M. W. A., & Zometa, P. (2025). Machine learning for EMG-based gesture recognition in brain–computer interfaces and humanoid robots. *International Journal of Intelligent Robotics and Applications*. <https://doi.org/10.1007/s41315-025-00463-1>
- Ben Abdallah, I., & Bouteraa, Y. (2024). An optimized stimulation control system for upper limb exoskeleton robot-assisted rehabilitation using a fuzzy logic-based pain detection approach. *Sensors*, 24(4), 1047. <https://doi.org/10.3390/s2404104>
- Capaldi, E. I. (2024). A low-cost wireless extension for object detection and data logging for educational robotics using the ESP-NOW protocol. *PeerJ Computer Science*, 10(1826). <https://doi.org/10.7717/peerj-cs.1826>
- Carvalho, C. R., Fernández, J. M., del-Ama, A. J., Barroso, F. O., & Moreno, J. C. (2023). Review of electromyography onset detection methods for real-time control of robotic exoskeletons. *Journal of NeuroEngineering and Rehabilitation*, 20(141). <https://doi.org/10.1186/s12984-023-01268-8>
- Hidalgo, M., Tene, G., & Sánchez, A. (2005). Fuzzy control of a robotic arm using EMG signals. *IEEE Proceedings*, Escuela Politécnica Nacional, Quito, Ecuador. <https://bibdigital.epn.edu.ec/bitstream/15000/9302/2/P0180.pdf>
- Nguyen, N. K., Dao, T. M. P., Nguyen, T. D., Nguyen, D. T., Nguyen, H. T., & Nguyen, V. K. (2024). An sEMG signal-based robotic arm for rehabilitation applying fuzzy logic. *Engineering, Technology & Applied Science Research*, 14(3), 14287–14294. <https://doi.org/10.48084/etasr.7146>
- Pérez-Juárez, J. G., García-Martínez, J. R., Medina Santiago, A., Cruz-Miguel, E. E., Olmedo-García, L. F., Barra-Vázquez, O. A., & Rojas-Hernández, M. A. (2024). Kinematic fuzzy logic-based controller for trajectory tracking of wheeled mobile robots in virtual environments. *Symmetry*, 17(2), 301. <https://doi.org/10.3390/sym17020301>
- Rahman, M. K. K., Nasor, M., & Imran, A. (2024). Enhanced hand gesture recognition with surface electromyogram and machine learning. *Sensors*, 24(16), 5231. <https://doi.org/10.3390/s24165231>

